Generative Artificial Intelligence

Redha Moulla

Paris, 5-6 November 2026

Course outline

- What is artificial intelligence?
- Fundamentals of machine learning and deep learning
- Large Language Models (LLMs)
- Prompting best practices
- RAG and agents
- Generative AI in the software development lifecycle
- Ethical, security and compliance issues in generative AI

What is artificial intelligence?

The literal meaning of artificial intelligence

1. Intelligence

The set of mental functions concerned with conceptual and relational knowledge.

- Larousse

2. Artificial

Produced by human activity, as opposed to nature.

- Larousse

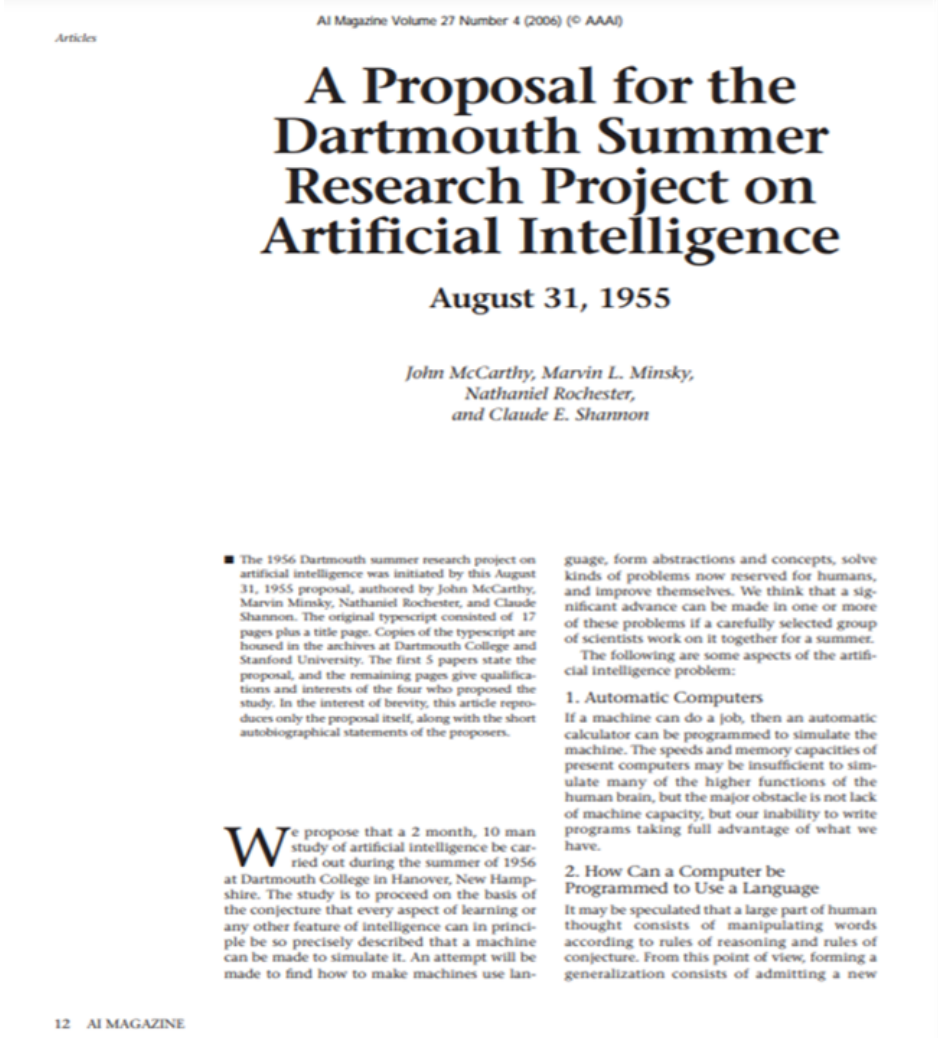
What is intelligence?

Intelligence encompasses several cognitive abilities:

1. **Reasoning:** The ability to solve problems and make logical deductions.
2. **Learning:** The ability to acquire new knowledge and improve through experience.
3. **Perception:** The ability to recognize and interpret sensory stimuli.
4. **Understanding:** The ability to grasp the meaning and significance of concepts and situations.
5. **Memory:** The ability to store and recall information.
6. **Creativity:** The ability to invent or produce new ideas and think in original ways.

But can intelligence be reduced to measurable abilities?

The Dartmouth Conference?



AI Magazine Volume 27 Number 4 (2006) (© AAAI)

Articles

A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence

August 31, 1955

John McCarthy, Marvin L. Minsky, Nathaniel Rochester, and Claude E. Shannon

■ The 1956 Dartmouth summer research project on artificial intelligence was initiated by this August 31, 1955 proposal, authored by John McCarthy, Marvin Minsky, Nathaniel Rochester, and Claude Shannon. The original typescript consisted of 17 pages plus a title page. Copies of the typescript are housed in the archives at Dartmouth College and Stanford University. The first 5 papers state the proposal, and the remaining pages give qualifications and interests of the four who proposed the study. In the interest of brevity, this article repeoduces only the proposal itself, along with the short autobiographical statements of the proposers.

We propose that a 2 month, 10 man study of artificial intelligence be carried out during the summer of 1956 at Dartmouth College in Hanover, New Hampshire. The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it. An attempt will be made to find how to make machines use language, form abstractions and concepts, solve kinds of problems now reserved for humans, and improve themselves. We think that a significant advance can be made in one or more of these problems if a carefully selected group of scientists work on it together for a summer.

The following are some aspects of the artificial intelligence problem:

1. Automatic Computers

If a machine can do a job, then an automatic calculator can be programmed to simulate the machine. The speeds and memory capacities of present computers may be insufficient to simulate many of the higher functions of the human brain, but the major obstacle is not lack of machine capacity, but our inability to write programs taking full advantage of what we have.

2. How Can a Computer be Programmed to Use a Language

It may be speculated that a large part of human thought consists of manipulating words according to rules of reasoning and rules of conjecture. From this point of view, forming a generalization consists of admitting a new

"We propose that a 2-month, 10-man study of artificial intelligence be carried out during the summer of 1956 at Dartmouth College in Hanover, New Hampshire. *The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it. An attempt will be made to find how to make machines use language, form abstractions and concepts, solve kinds of problems now reserved for humans, and improve themselves. We think that a significant advance can be made in one or more of these problems if a carefully selected group of scientists work on it together for a summer."*

Artificial intelligence according to John McCarthy

“It is the science and engineering of making intelligent machines, especially intelligent computer programs. It is related to the similar task of using computers to understand human intelligence, but AI does not have to confine itself to methods that are biologically observable.”

- John McCarthy

The Turing Test

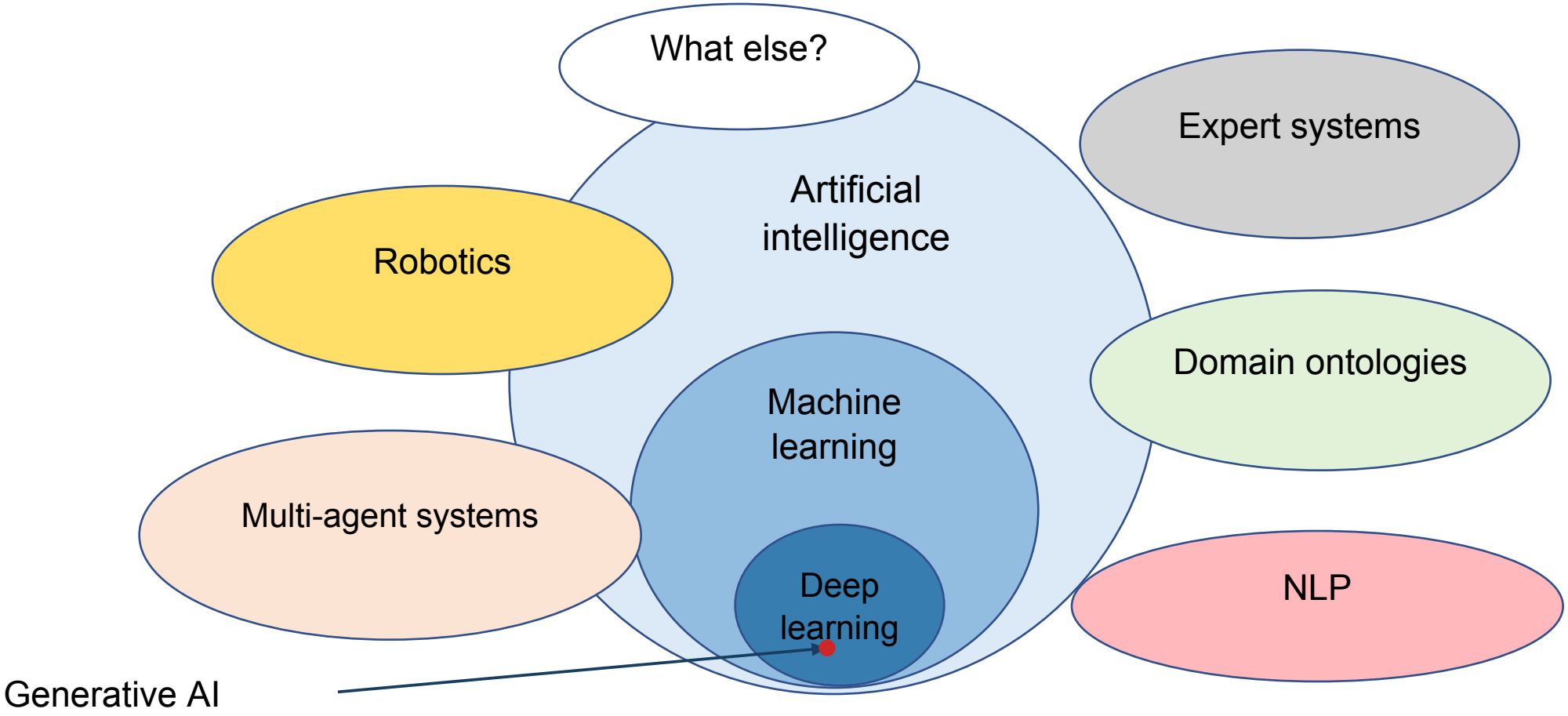
Developed by Alan Turing in 1950, the Turing Test attempts to assess a machine's intelligence, or more precisely its ability to think. Because this is difficult to measure, the test substitutes the ability to process natural language like a human for the ability to think.

Key features of the Turing Test:

- A human interrogator converses with a human and a machine, both hidden from view.
- If the interrogator cannot consistently identify the machine, it is considered to have passed the test.
- The test does not measure knowledge or truthfulness, but the ability to reproduce human behaviour.

A pragmatic definition of artificial intelligence

A set of techniques that enable machines to perform tasks traditionally requiring human intelligence.



Strong AI vs weak AI

The distinction between strong AI and weak AI refers to two different conceptual approaches to artificial intelligence.

Weak AI:

- Also known as “narrow” AI, it is designed to perform specific tasks and has no consciousness.
- Weak AI systems act and react only according to programmed instructions and specific algorithms.
- Examples: Virtual assistants, recommendation systems and speech recognition.

Strong AI:

- Aims to create machines with generalization abilities similar to human intelligence.
- Strong AI would be able to learn, reason, solve problems and make decisions independently.
- Strong AI remains a goal to be achieved and a subject of intensive research.

Fundamentals of machine learning and deep learning

What is machine learning?

Machine learning is a branch of artificial intelligence that enables machines to learn from data without being explicitly programmed to perform specific tasks.

- **Discriminative AI:** Models the boundary between classes.
Examples: Classifying customer comments, routing emails, credit scoring, medical diagnosis, etc.

- **Generative AI:** Models the data distribution to generate new data.
Examples: Generating text, images, music, etc.

- **Reinforcement learning:** Learns to make sequential decisions in an environment.
The agent does not imitate or classify: it acts to maximize a reward.
Examples: Bidding algorithms, video games, etc.

Discriminative machine learning

Machine learning is a branch of artificial intelligence that enables machines to learn from data without being explicitly programmed to perform specific tasks.

Machine learning encompasses two broad types of learning:

- **Supervised:** Algorithms learn from labelled data to make predictions or classifications.
- **Unsupervised:** Learning uses unlabelled data to discover hidden structures.

Supervised learning

Supervised learning involves learning a model that maps a set of *features* to a *label*.

• **Inputs:** A labelled dataset used to train the model.
Example: Texts (tweets, etc.) labelled with positive or negative sentiment.

• **Output:** A label for a data point the model has not seen.

Supervised learning itself has two main types:

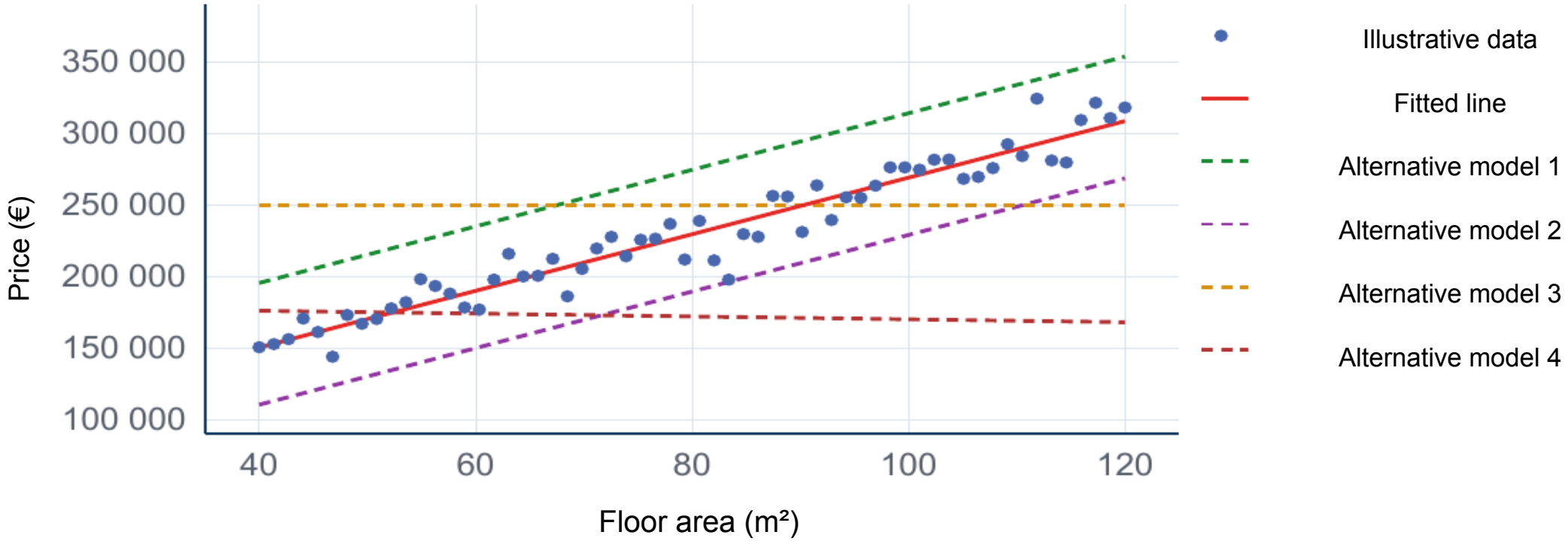
• **Classification:** Predicting a category or class.
Examples: An image label (cat, dog, etc.), the sentiment of a text, or a customer's interests inferred from their comments.

• **Regression:** Predicting a continuous value, typically a real number.
Examples: The price of an apartment or a customer's lifetime value.

Simple linear regression

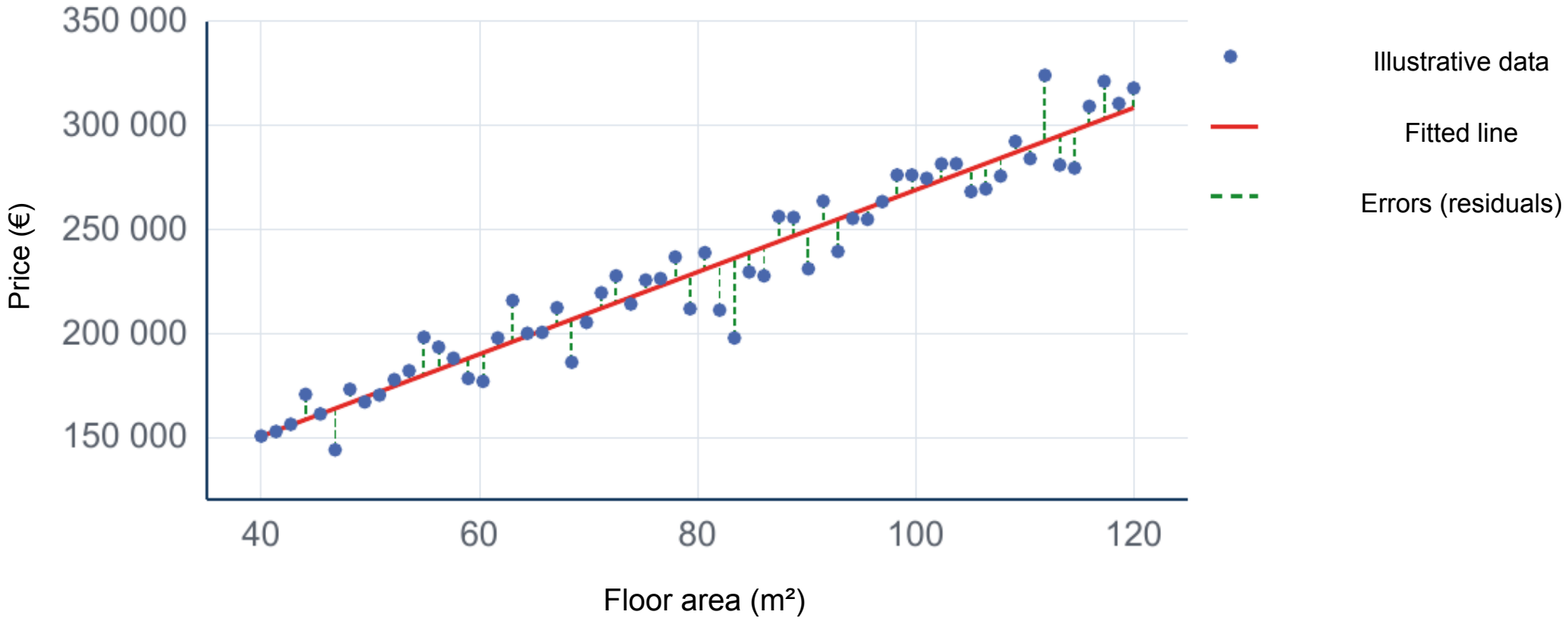
Given *n* observations $x_1, x_2, ..., x_n$ with corresponding labels $y_1, y_2, ..., y_n$, we seek the linear model that best fits the data.

$$\hat{y} = \beta_0 + \beta_1 x$$

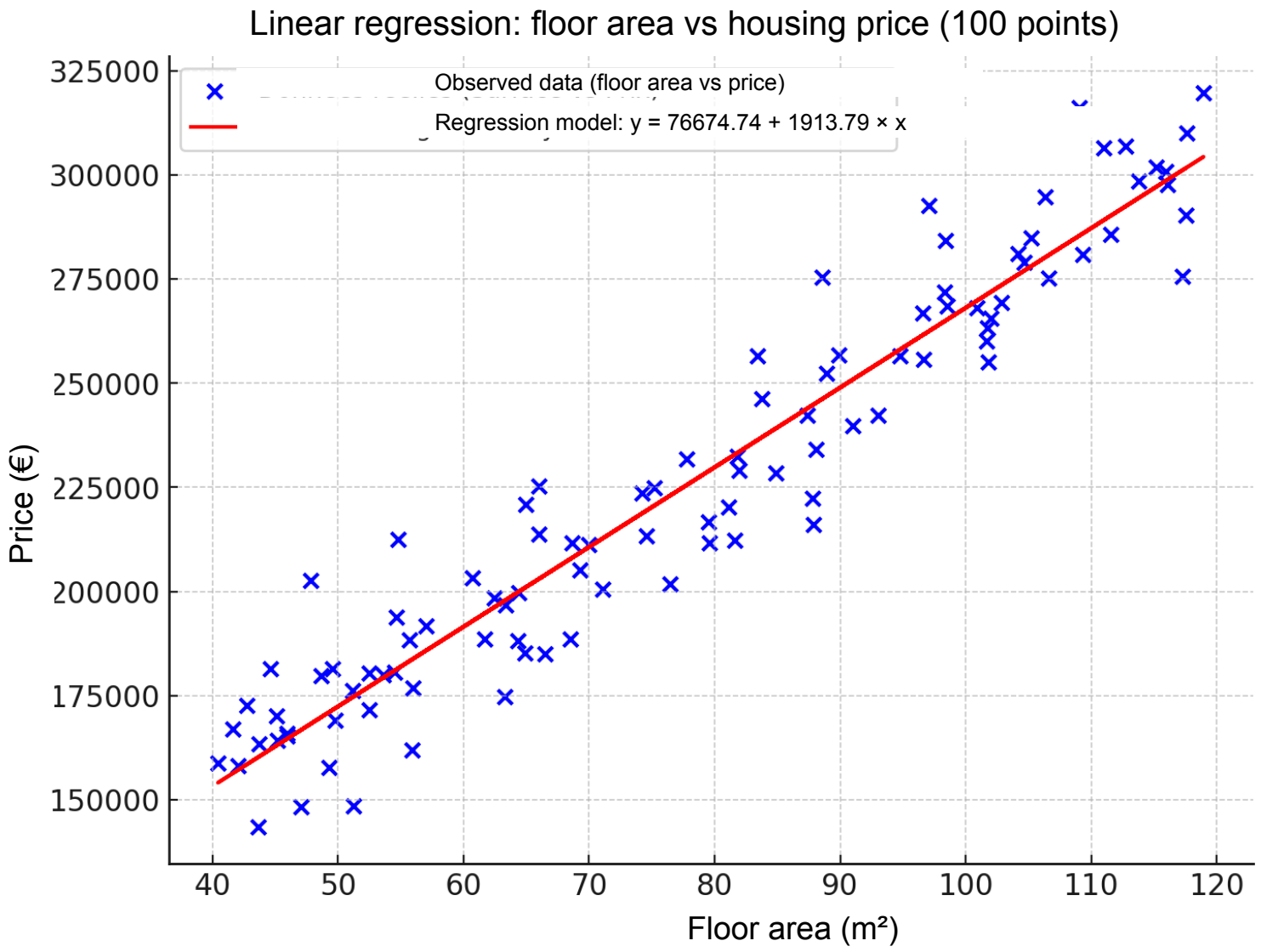


Empirical risk minimization 1/2

The prediction error for the *i*-th observation is $e_i = y_i - \hat{y}_i$, where $\hat{y}_i = \beta_0 + \beta_1 x_i$.



Predicting price from floor area: inference



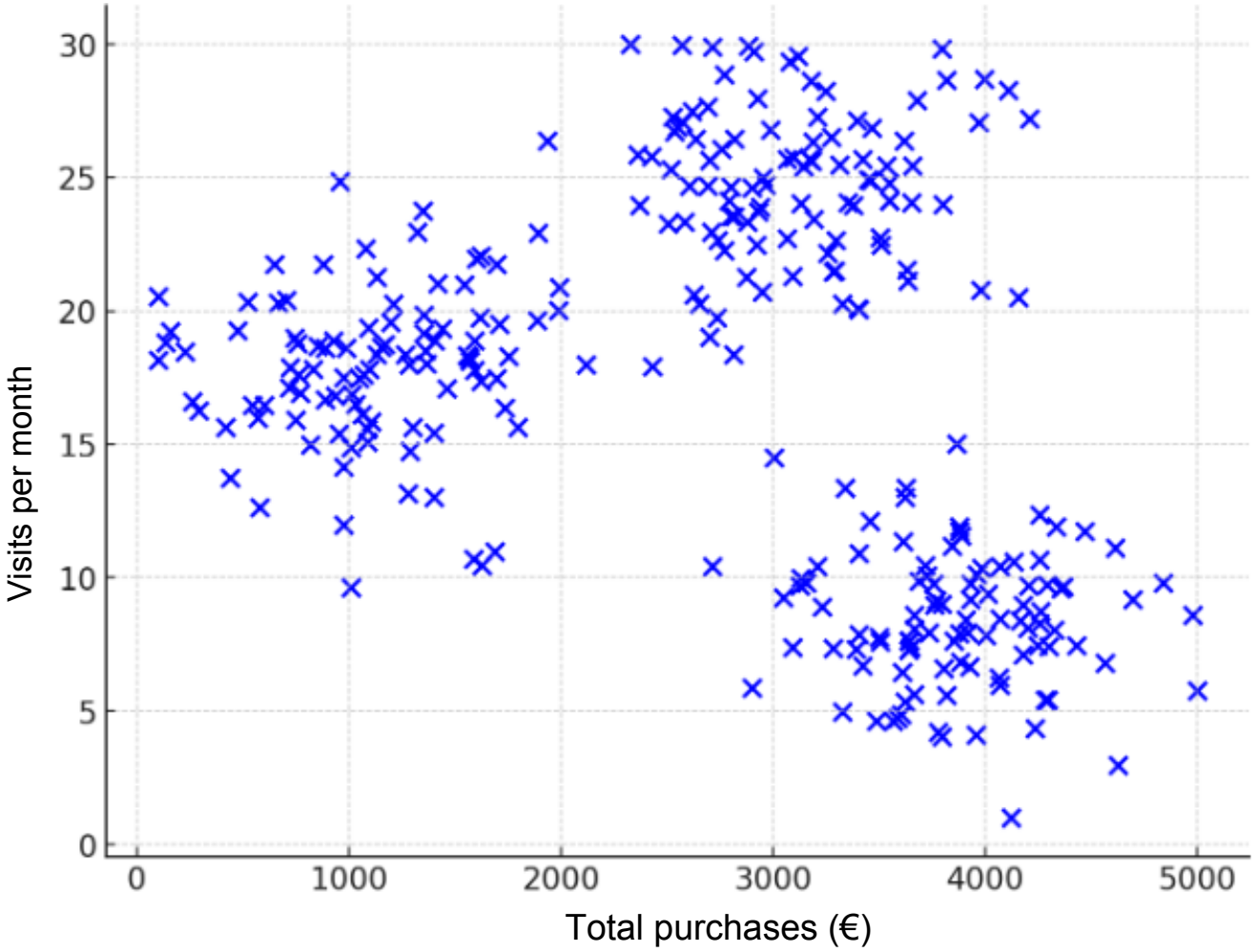
Unsupervised learning

Unsupervised learning uses machine learning models to identify patterns and structures in unlabelled data.

Main types of unsupervised learning:

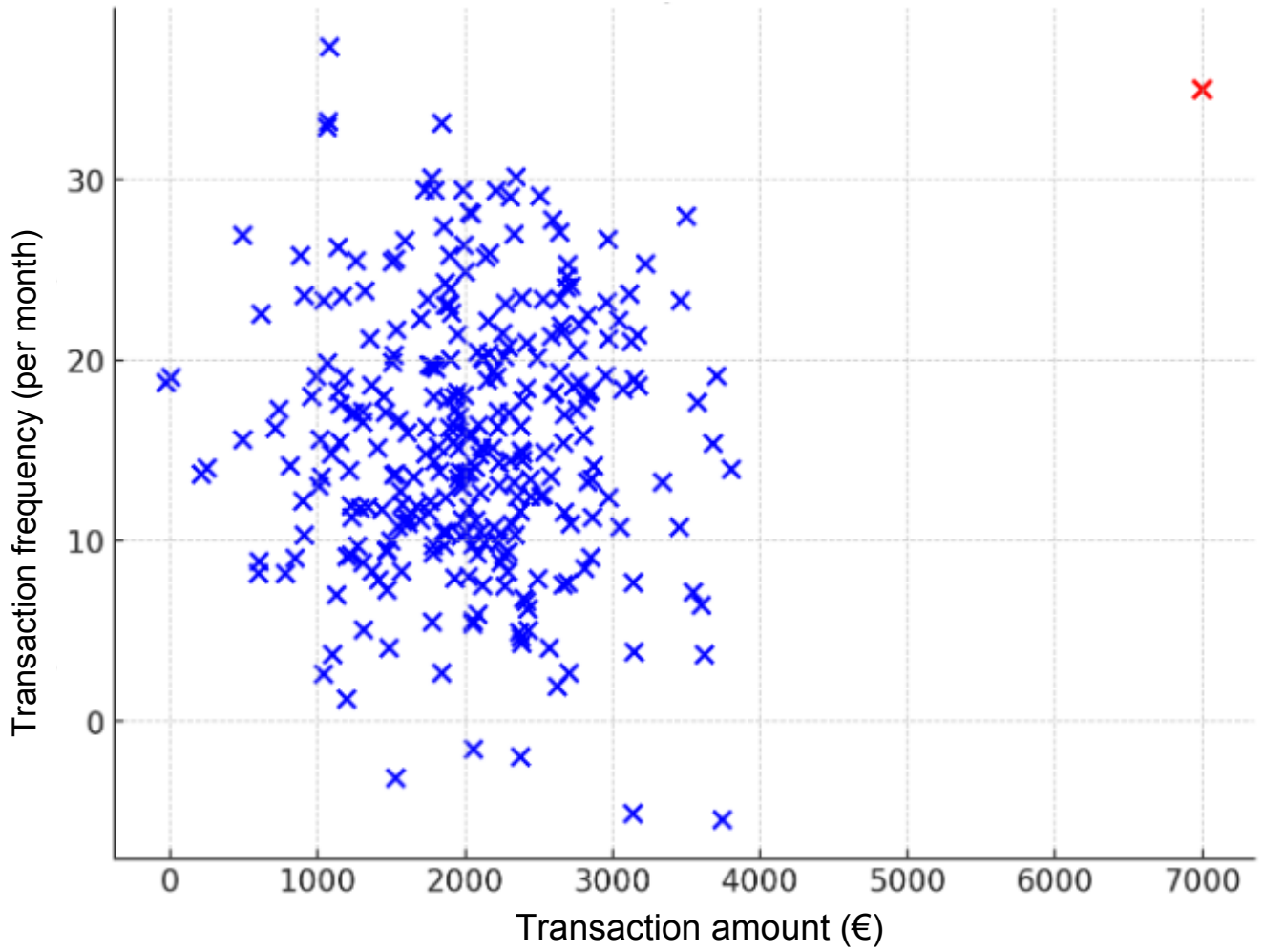
- **Clustering:** Grouping similar data points together. Examples: Market segmentation and social grouping.
- **Anomaly detection:** Detecting observations whose characteristics are unusual compared with the majority.

Clustering example: customer segmentation



Anomaly detection

Anomaly detection example: bank fraud



Fundamentals of deep learning

A brief history of deep learning

1940-1980: Foundations

• 1943: Artificial neuron (McCulloch & Pitts).

• 1986: Backpropagation (Rumelhart, Hinton, Williams).

1990-2000: Winter

• Lack of data and computing power.

• Advances in theory: SVMs and Bayesian models.

2010-2020: Renaissance

• 2012: AlexNet revolutionizes computer vision.

Since 2020: Multimodality and LLMs

• 2017: The Transformer, the foundation of LLMs.

• 2018-2020: GPT-3, BERT.

• 2022: ChatGPT...

Introduction to neural networks

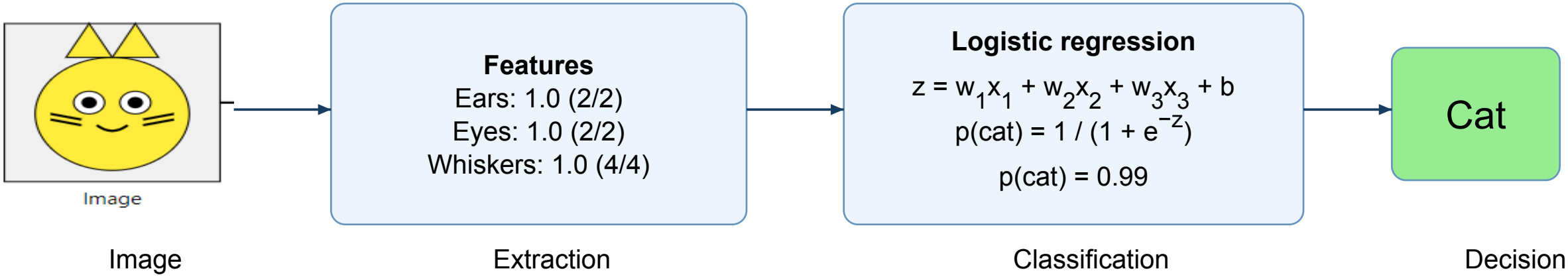
Definition: Neural networks are computational models inspired by how neurons work in the human brain. They can learn complex tasks by modelling nonlinear relationships between inputs and outputs.

Characteristics:

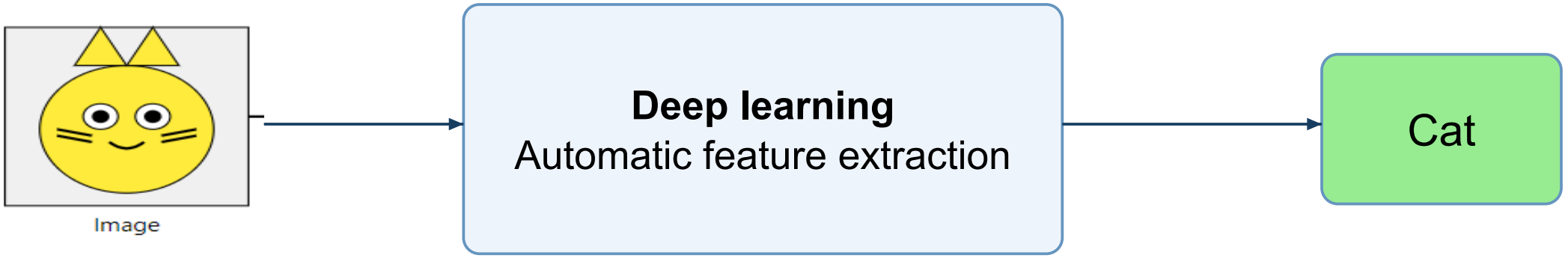
- **Automatic feature extraction:** Ability to adapt and extract features from data without explicit programming.
- **Nonlinear modelling:** Ability to capture complex relationships in data.
- **High-dimensional modelling:** Deep learning models are particularly well suited to high-dimensional data (images, text, etc.).
- **Flexibility:** Applicable to a wide range of tasks and data types (images, natural language, graph data, etc.).

Feature extraction

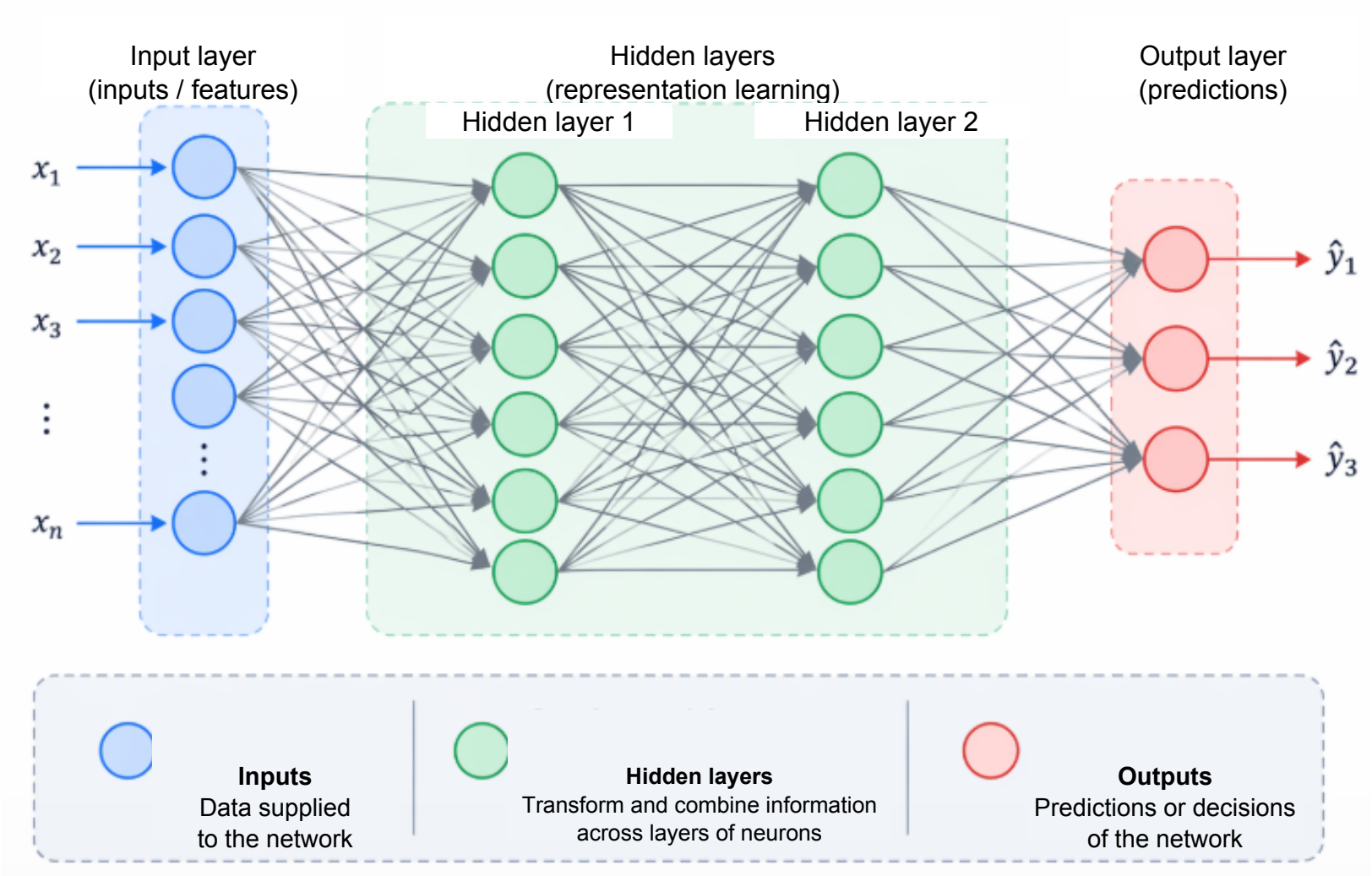
Feature extraction in “traditional” machine learning

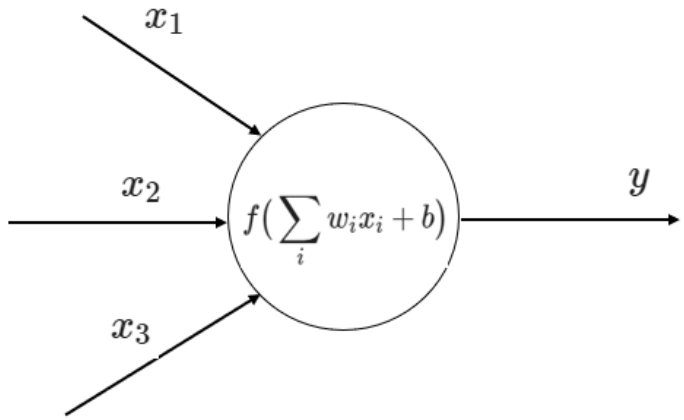


Feature extraction in deep learning



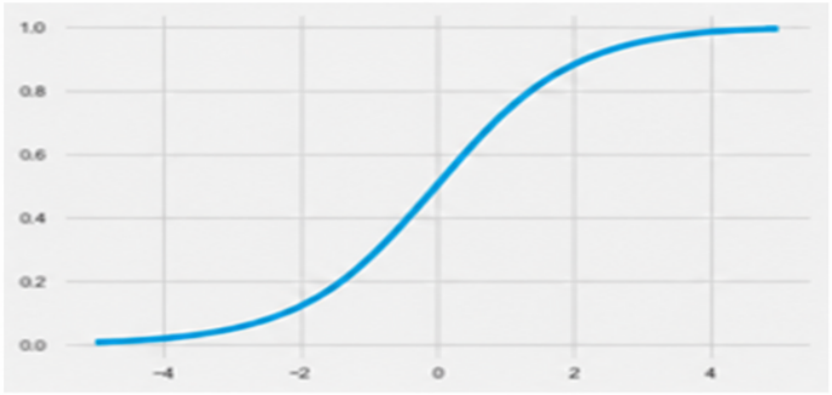
Neural network and artificial neuron



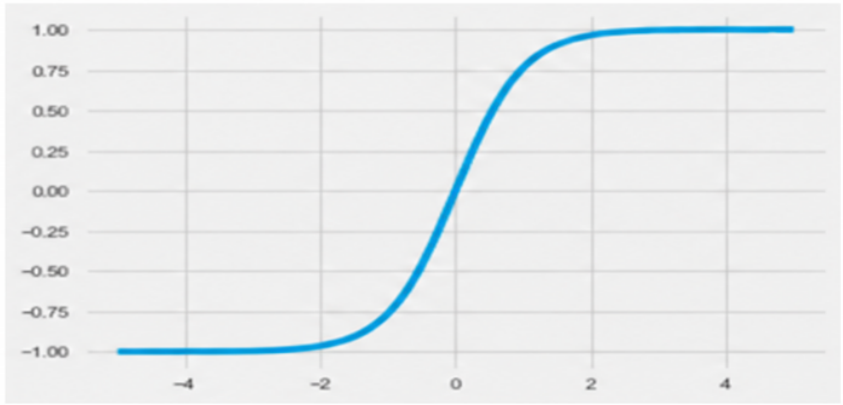


Illustrating activation functions

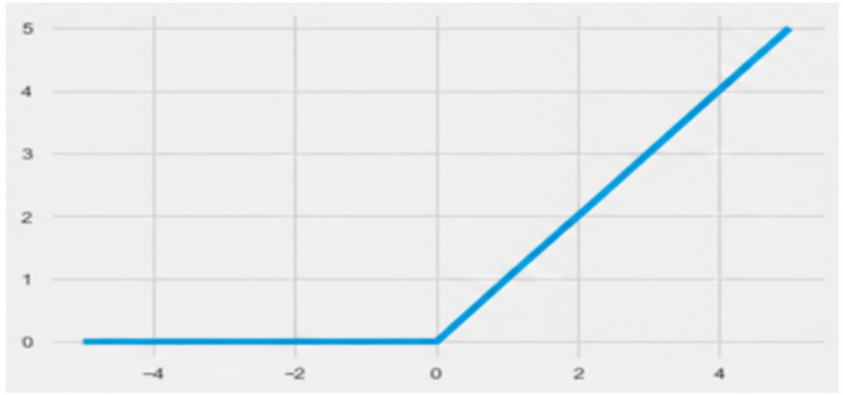
Sigmoid



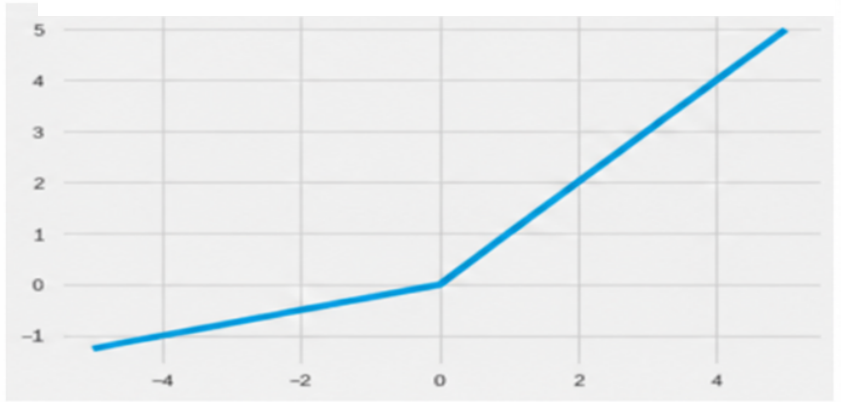
Tanh



ReLU



Parametric ReLU



Softmax in the final classification layer

Softmax transforms a list of scores into **class probabilities**. It is the final step in a multiclass classification model.

$$P(y_i) = \frac{\exp(s_i)}{\sum_{j=1}^{K} \exp(s_j)}$$

Here, s_i is the score associated with class *i*, and K is the total number of classes.

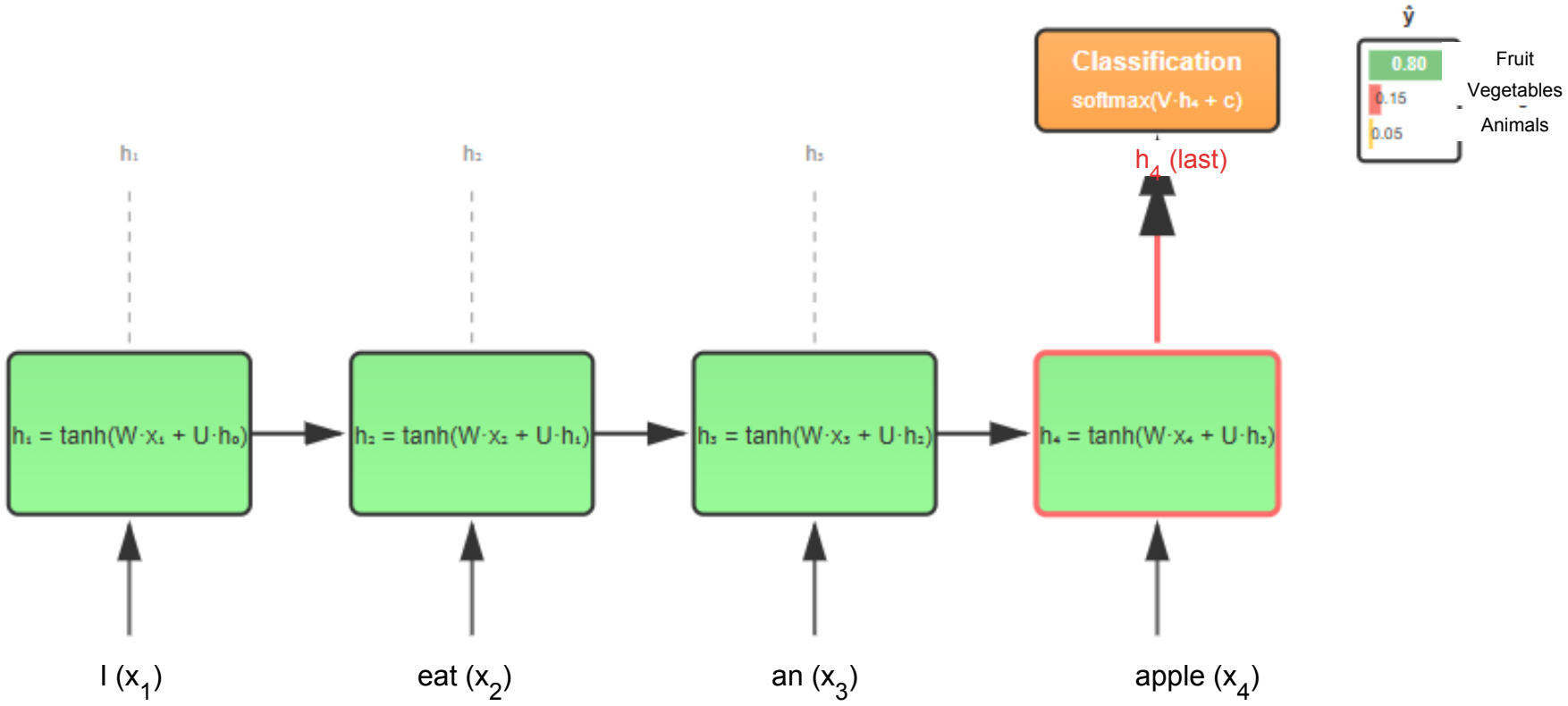
Example: Image → cat: 0.80 | dog: 0.10 | sheep: 0.10

The model predicts the most likely class (**cat**), while retaining estimates for the other classes.

Large Language Models (LLMs)

Earlier language processing models: RNNs

- **Short-term memory:** Recurrent neural networks (RNNs) can “remember” past information through their recurrent connections.
- **Sequence processing:** They are particularly suited to tasks involving sequential data where context matters.



Transformers

arXiv:1706.03762v7 [cs.CL] 2 Aug 2023

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez*† University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin*‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

†Work performed while at Google Brain.

‡Work performed while at Google Research.

31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.

The original Transformer

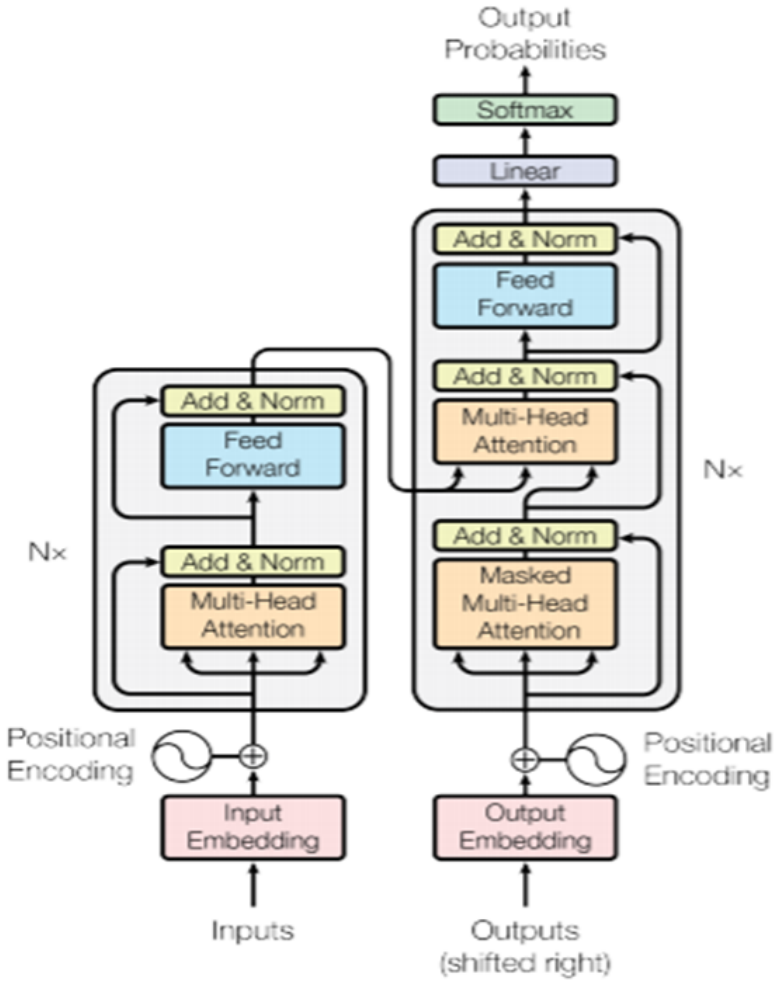
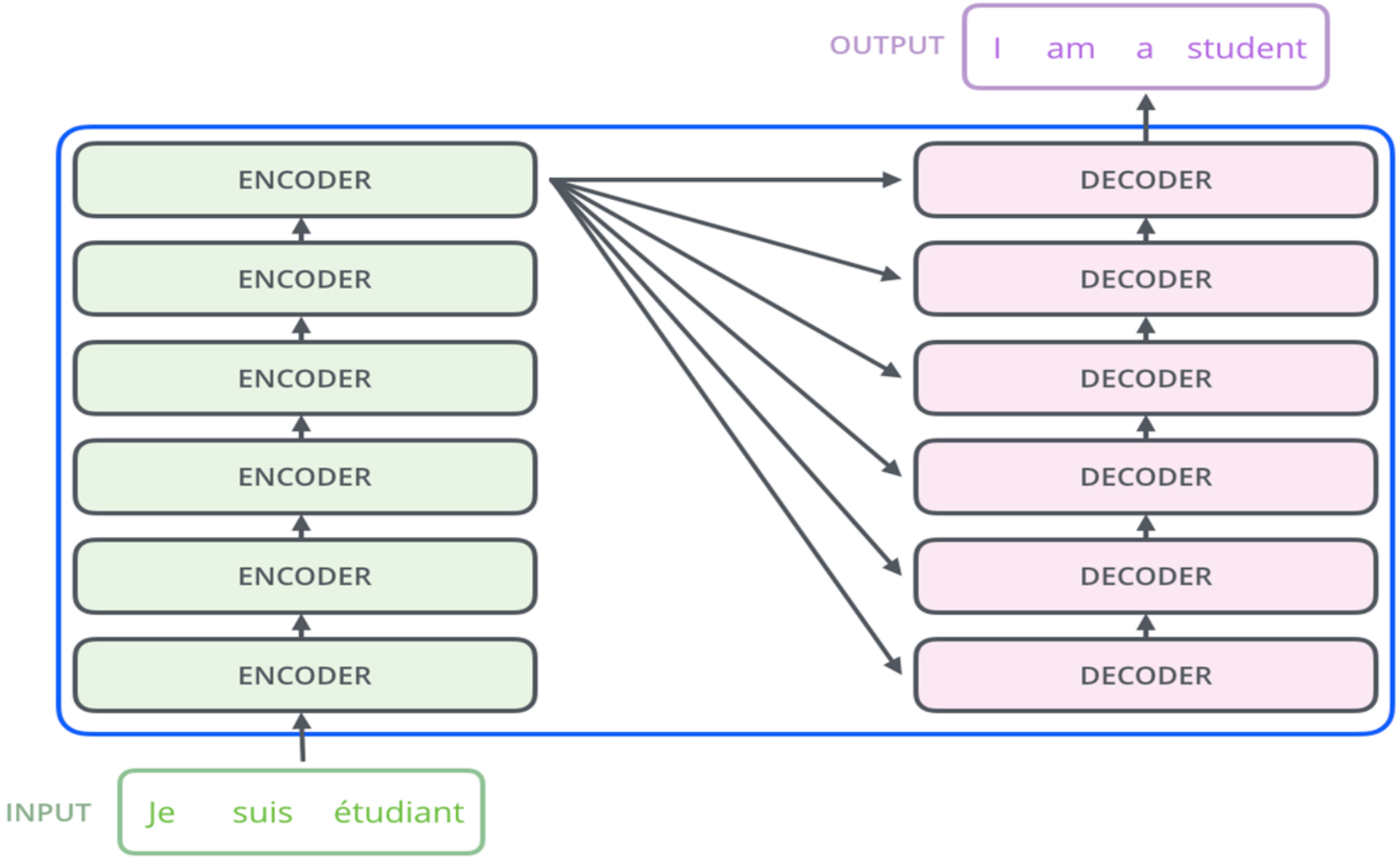
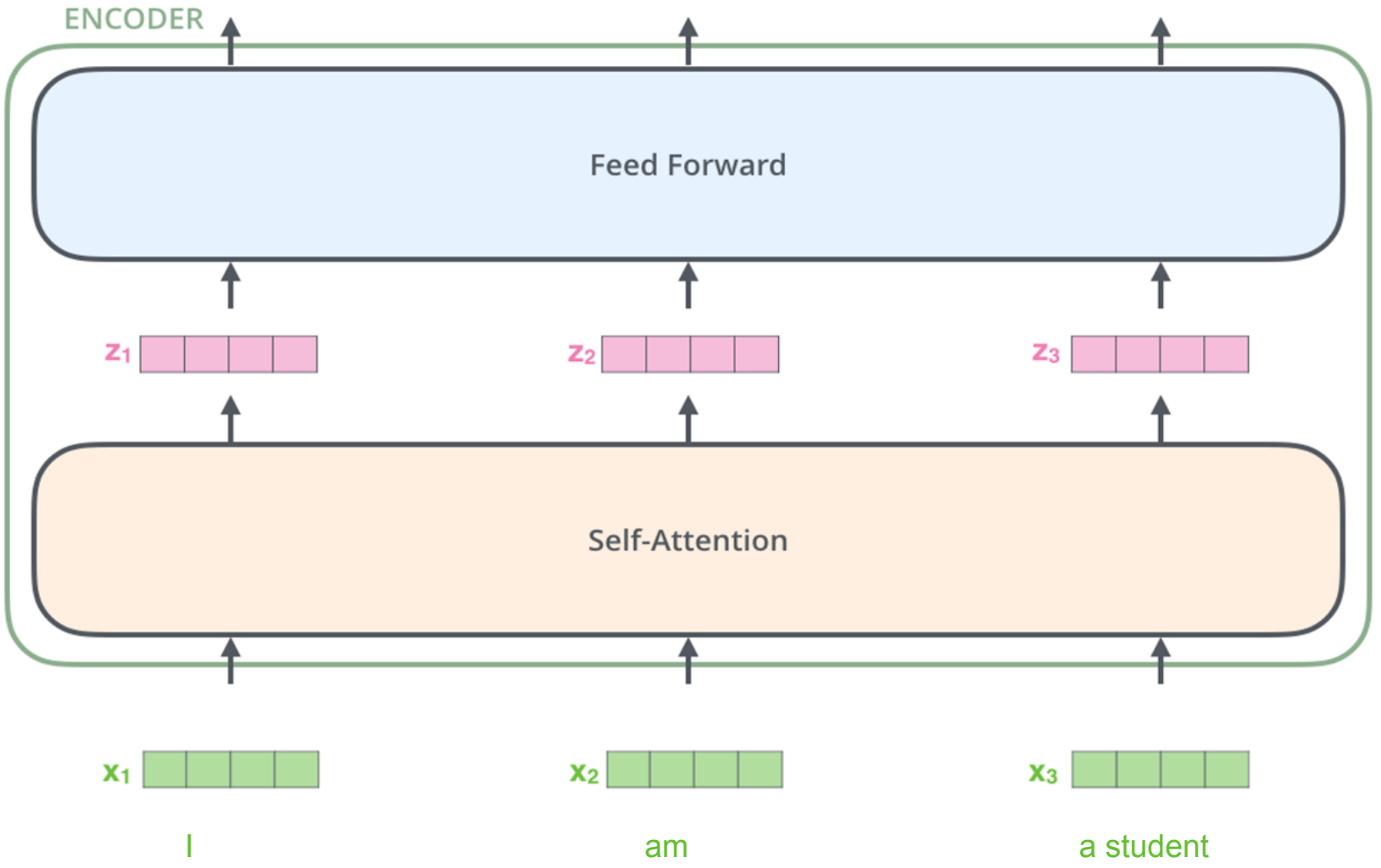


Figure 1: The Transformer - model architecture.

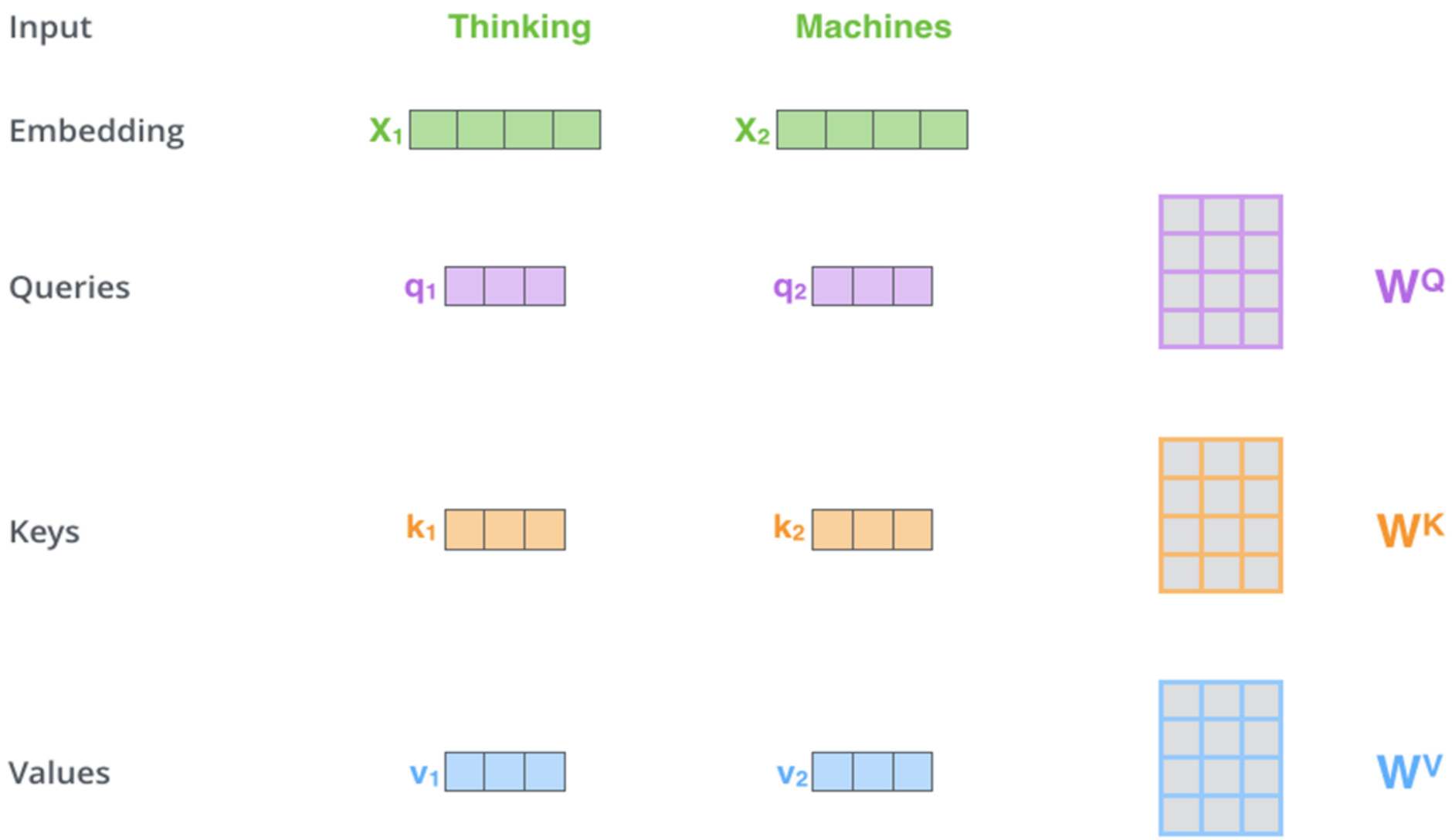
Cross-attention mechanism



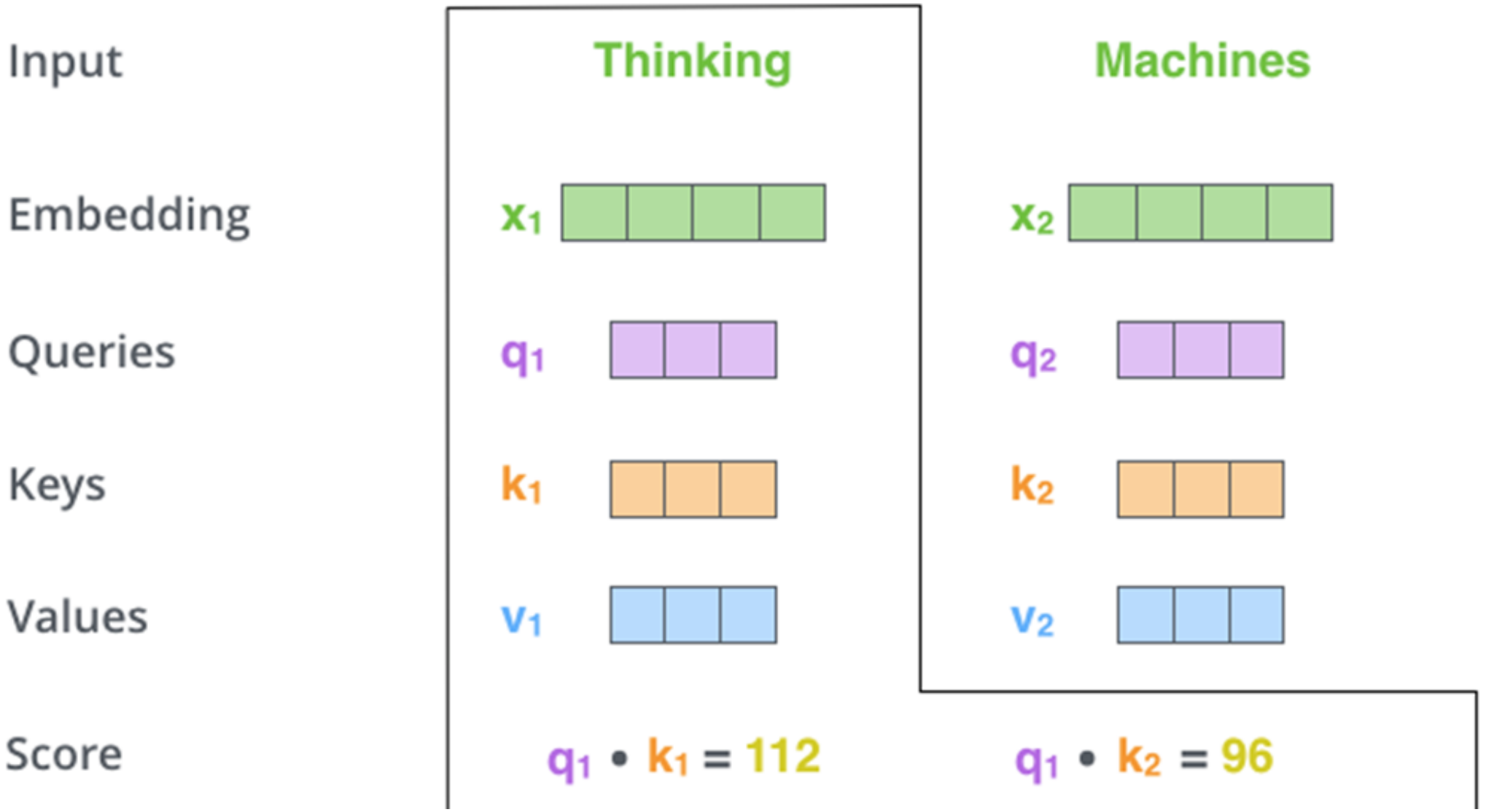
Self-attention mechanism



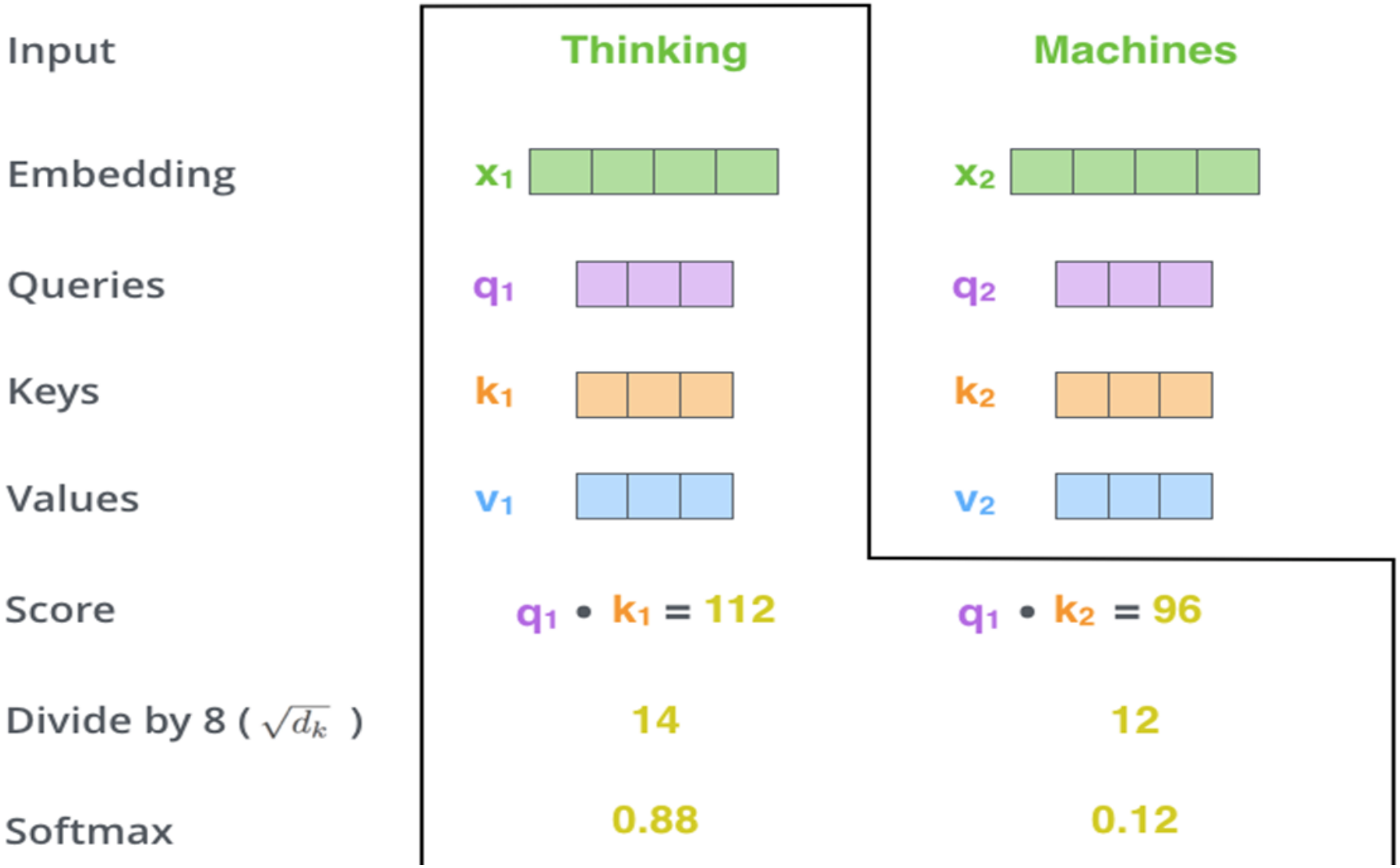
How self-attention works



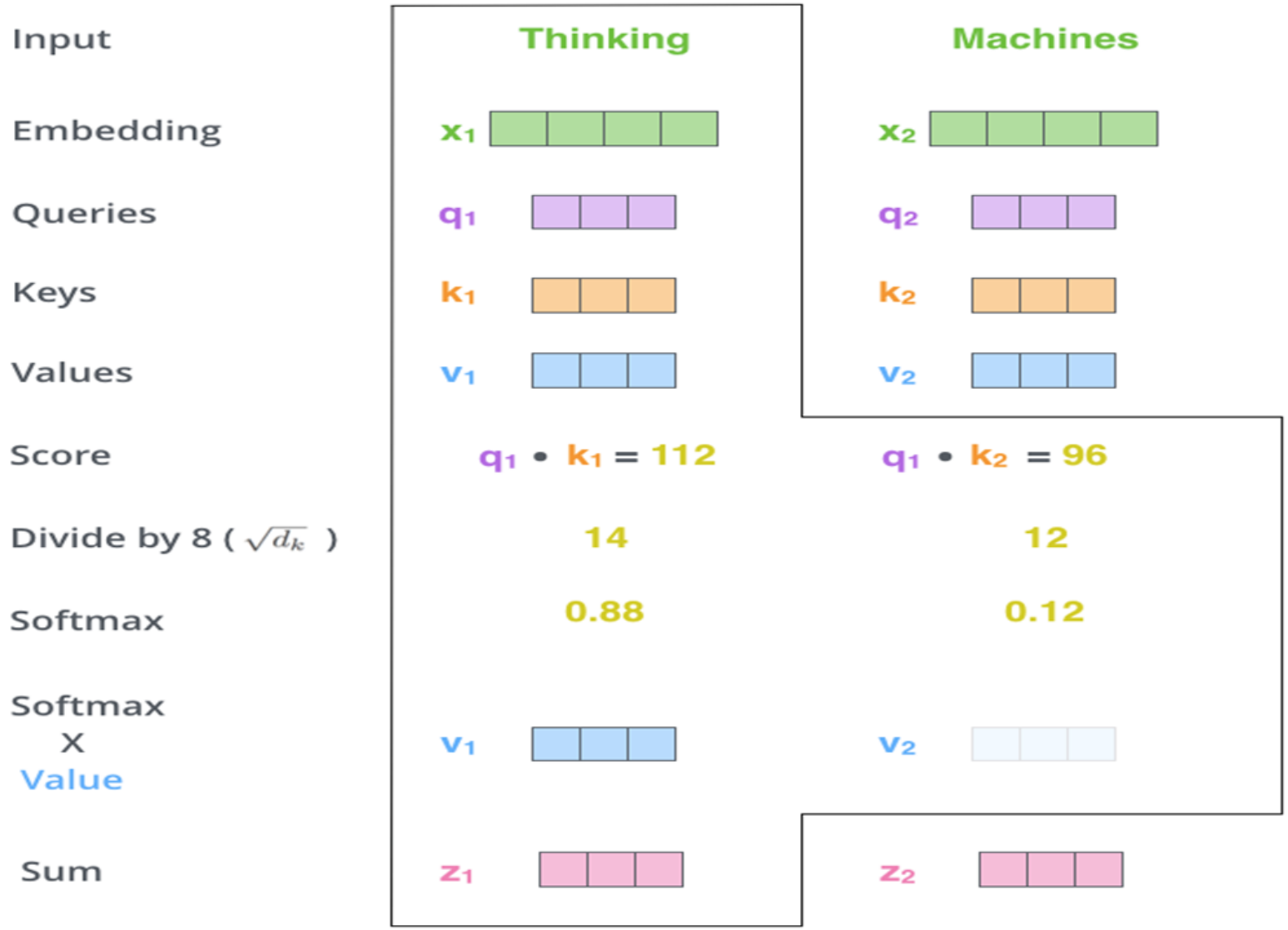
Calculating self-attention scores



Calculating self-attention scores



Computing the new representation



Performance of the original Transformer

Trained on the WMT 2014 English-German dataset, with around 4.5 million sentences, and the WMT 2014 English-French dataset, with around 36 million sentences.

- 8 NVIDIA P10 GPUs
- Base: 12 hours
- Large: 3.5 days

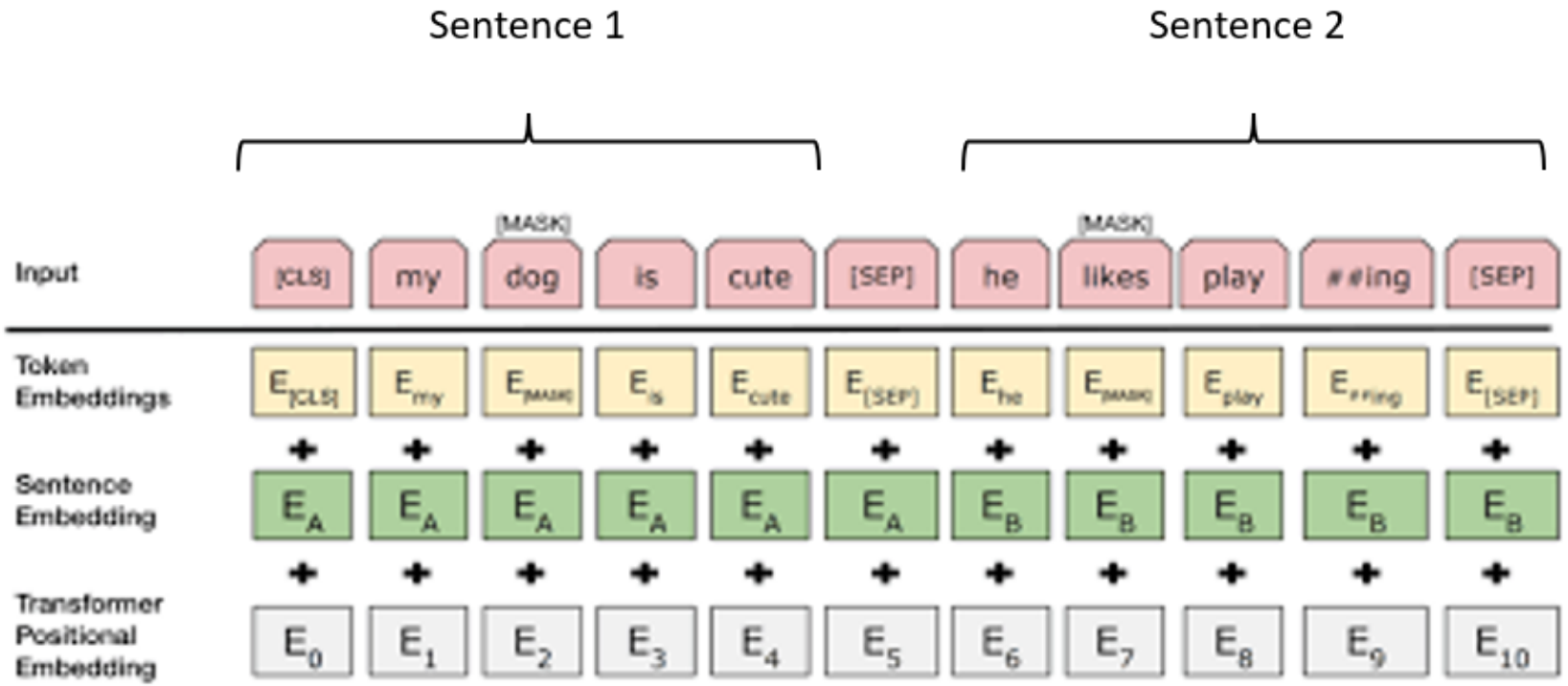
Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	**$3.3 \cdot 10^{18}$**	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

BERT

BERT (Bidirectional Encoder Representations from Transformers) represents a major advance in NLP, introducing a new approach to language modelling. Its bidirectionality captures the context of words, enabling a more nuanced understanding of language.

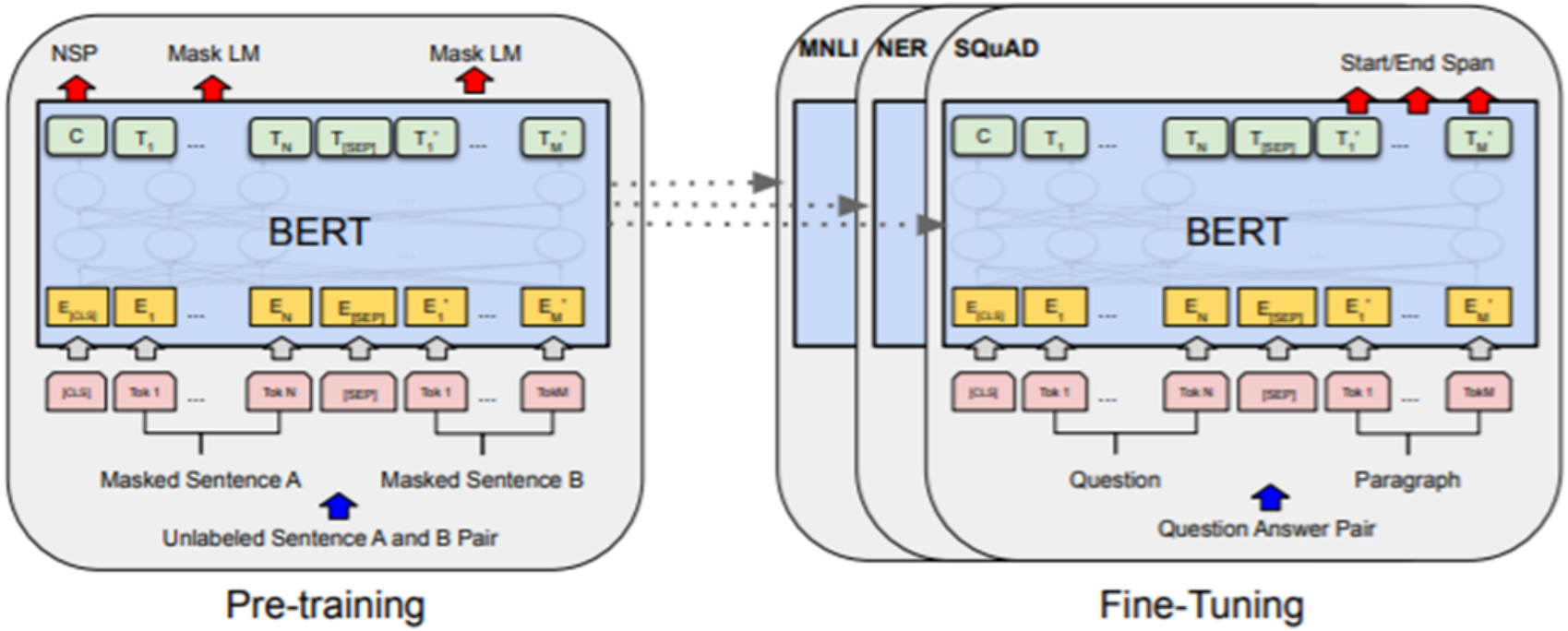
Pre-training



Fine-tuning BERT for specific tasks

After pre-training, BERT is fine-tuned for specific tasks:

- Add a task-specific output layer (classification, NER, QA).
- Fine-tune all parameters of the pre-trained model on the task's corpus.



BERT performance

BERT was pre-trained on BookCorpus (800 million words) and English Wikipedia (2.5 billion words).

Two models:

- **BERT Base:** 12 layers and 110 million parameters.
- **BERT Large:** 24 layers and 340 million parameters.

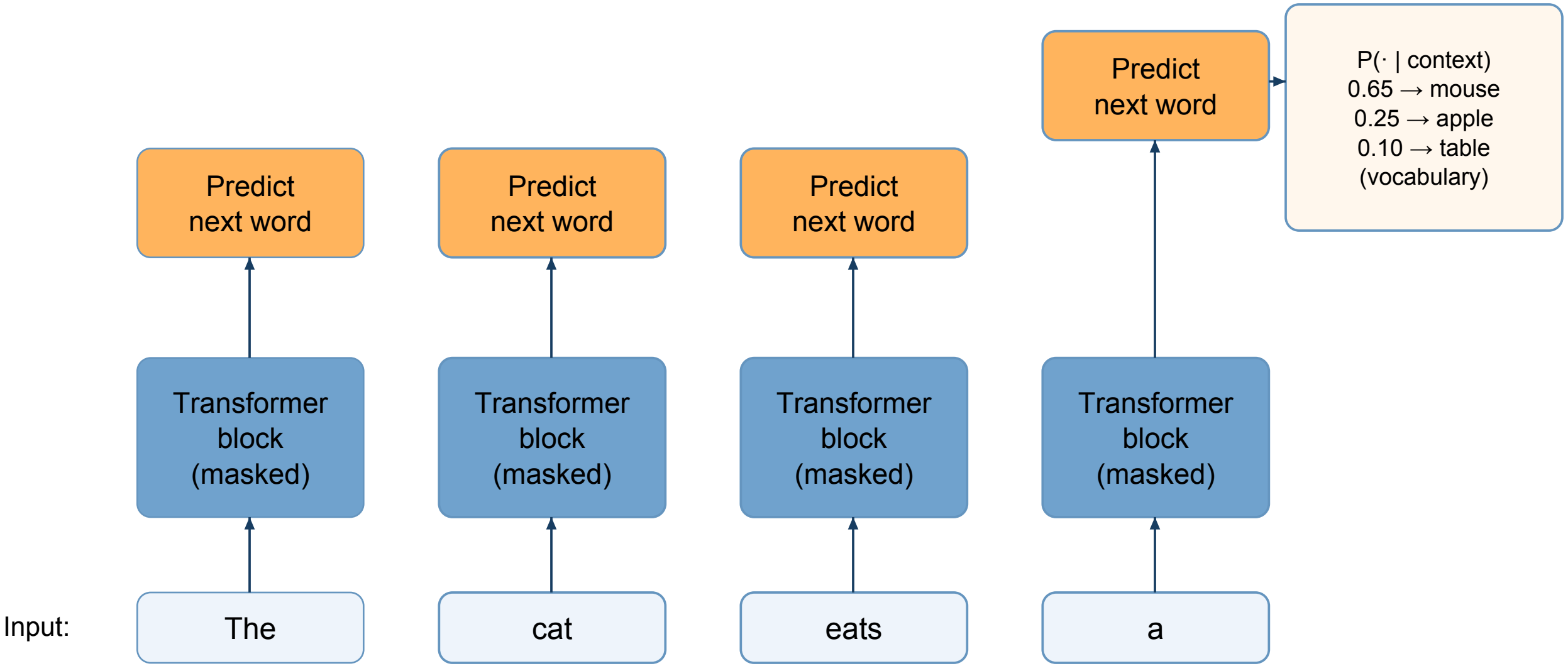
System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$BERT_{BASE}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$BERT_{LARGE}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

Generative models (GPT)

“What I cannot create, I do not understand.”

- Richard Feynman

How generative models work



The temperature parameter

Definition: Temperature (T) controls randomness in generated responses. It affects the probability distribution of words before sampling.

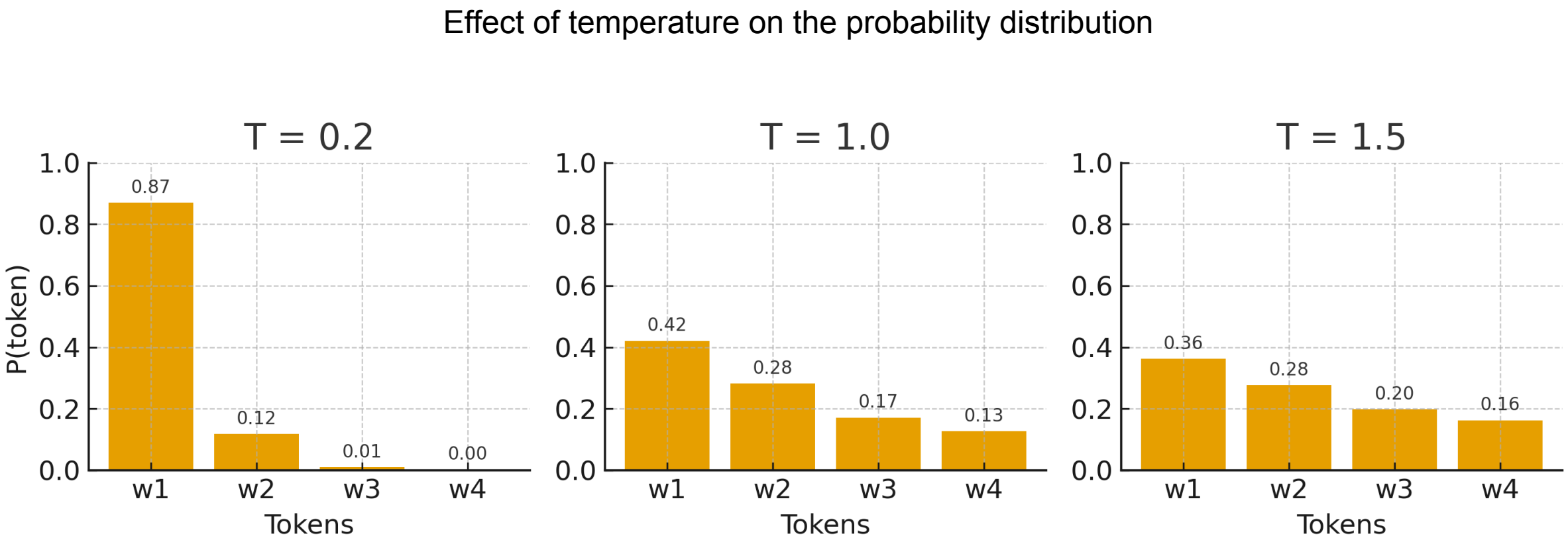
$$P(x_i) = \frac{\exp(z_i / T)}{\Sigma_j \exp(z_j / T)}$$

- T < 1 → a more concentrated distribution and more predictable responses.
- T > 1 → a flatter distribution and more varied, creative responses.

In other words:

- Low temperature → precision and consistency.
- High temperature → imagination and diversity.

The effect of temperature: a graphical illustration



Tokenization

The initial problem: Language models cannot directly process raw text: there are too many words, accents, punctuation marks and variations (millions of possible combinations).

Tokenization: Splits text into a limited number of reusable units called **tokens**. Each token is represented by a numerical identifier that the model can learn to predict.

Why tokenize?

- Reduce vocabulary complexity: from millions of words to tens of thousands of tokens.
- Handle unknown or rare words by breaking them down into known subunits.
- Provide a uniform numerical representation of language that neural networks can process.

What model training data contains

General principle: Language models are trained on enormous text corpora, often trillions of tokens, drawn from diverse sources to cover language, facts and styles.

Typical data sources:

- **Public web:** Web pages, articles, forums and Wikipedia (e.g. Common Crawl).
- **Books and documents:** Literary works, scientific papers and technical reports.
- **Code:** GitHub repositories, notebooks and technical documentation.
- **Conversations and dialogues:** Chat datasets, forum discussions and human-annotated data.
- **Synthetic data:** Text generated by other models to enrich particular topics.

Desired balance:

- Linguistic and topical diversity (languages, registers and domains).
- Quality and consistency (filtering spam, duplicates and toxic content).
- Balanced representation of everyday, technical and conversational language.

GPT-3

- Developed by OpenAI; released in 2020.
- Autoregressive model based on the Transformer architecture.

Characteristic	Value / detail
Date	2020
Parameters	175 billion
Training corpus	Around 300 billion tokens
Maximum context	2,048 tokens
GPU resources	Several hundred GPUs (V100 type)
Estimated training cost	$4.6M-$12M, depending on the source

- Applications: Text generation, Q&A, summarization, translation and coding assistance.

Few-shot learning

- Large models such as GPT-3 can perform tasks **without explicit retraining** on those tasks.
- The approach uses a few labelled examples directly in the prompt or input.
- **Zero-shot:** No demonstrations are provided; the model must infer the task from its general knowledge.
- **One-shot / few-shot:** One or a small number of examples guide the model in solving the task.

Example:

English: “cat” → French: “chat”
English: “house” → French: “maison”
English: “car” → French: “voiture”
English: “tree” → French:

ChatGPT

ChatGPT, developed by OpenAI, is a language model based on the GPT (Generative Pre-trained Transformer) architecture and optimized to understand and generate natural dialogue. Its training involves the following stages:

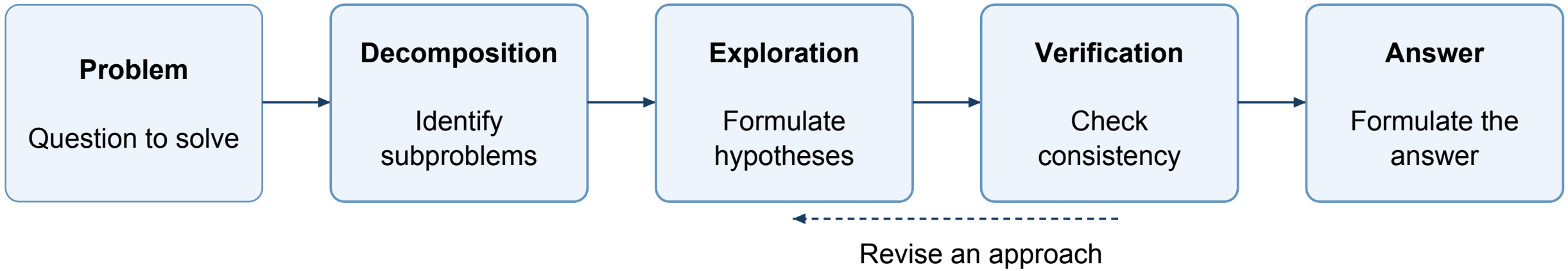
1. **Pre-training on a large corpus:** Like GPT-3, ChatGPT is first pre-trained on a large text dataset covering a wide range of material available online, to learn a general understanding of language.

2. **Supervised fine-tuning:** ChatGPT is then fine-tuned on specific dialogues to improve its conversational skills. Question-answer pairs and conversations teach dialogue structures and contextually appropriate responses.

3. **Reinforcement Learning from Human Feedback (RLHF):** Reinforcement learning adjusts the model's responses using preferences and corrections from human evaluators, further improving relevance and naturalness.

Reasoning models

These models are trained to break down a problem, explore possible approaches and check their results before answering.



More computation can improve performance on complex problems, at the cost of longer response times.

Proprietary vs open-source LLMs

Proprietary models

- ChatGPT, Claude, Gemini, Grok...
- Weights are not publicly available.
- High performance; access restricted to paid APIs.
- Strict intellectual property protection; code is not accessible.
- Technical support and extensive documentation.
- Vendor dependence and potentially high costs.

Open-source models

- Llama, Mistral, Gemma, DeepSeek, Qwen...
- Publicly available weights.
- Code transparency and flexibility to adapt models.
- Sometimes lower performance, but continuous improvement.
- Free to obtain, but infrastructure costs must be managed.
- Exposure to potential security vulnerabilities; community support.

Conclusion: Choose between control, flexibility and optimal performance.

Introduction to prompting

Definition: Prompting is the art of writing instructions for a language model to obtain a relevant, high-quality response.

Key idea: Output quality depends heavily on how the input is written.

Goals:

- Obtain accurate, consistent results.
- Reduce ambiguity and hallucination risks.
- Adapt the response's style and level of detail.

Basic prompting techniques

- **Clear, explicit prompts:** Avoid vague wording.
- **Context:** Provide additional information to guide the model.
- **Expected format:** Specify the desired output structure (list, table, short text).
- **Examples:** Show one or more cases to guide generation.

Example: Instead of “What is machine learning?”, write “Give a detailed explanation of how machine learning works, illustrated with an example.”

1. Tone changes the structure of reasoning

Principle: A prompt's tone influences not only style, but also the logical structure of the response.

Observed effects

• A calm, analytical tone produces orderly, deductive writing: the model explains causes and develops ideas before concluding.

• An urgent or tense tone produces terse, action-oriented writing, with short sentences and lists of decisions.

Examples

"Explain calmly, following a logical line of reasoning."
"Quickly: summarize what I need to tell the client in two minutes."

Takeaway: Tone is a cognitive lever: it changes how the model reasons before it even starts writing.

2. Metaphors frame thinking

Principle: A metaphor imposes a reasoning framework, prompting the model to recast a concept in a familiar structure.

Observed effects

• Abstract concepts become accessible through analogy.

• The response naturally follows the dimensions of the metaphor (for example: ingredients, recipes and cooking).

Examples

“Explain machine learning as if it were a kitchen.”
“Explain optimization as a hike with stages and peaks.”

Takeaway: A metaphor acts as a cognitive framework that guides the text’s logic and vocabulary.

3. Roles activate latent corpora

Principle: Assigning the model a role selects a way of speaking and reasoning drawn from its corpus.

Observed effects

• Historian: Emphasizes chronology and underlying causes.

• Sceptical engineer: Focuses on assumptions, tests and limitations.

• Critical poet: Favours imagery, tensions and rhythm.

Examples

“Act as a historian and explain the underlying causes.”
“Answer as an engineer who questions their own assumptions.”

Takeaway: A role acts as a stylistic and cultural filter, guiding the response without changing its substance.

4. Precise context matters more than length

Principle: Specific, concrete context produces better responses than a long list of abstract instructions.

Observed effects

• The model understands the situation better when it is grounded in a concrete setting.

• Responses become more relevant and natural.

Examples

“You are writing to your manager. Use a firm but respectful tone. Your goal is to obtain approval before 6 p.m.”
“In 200 words, identify three risks and formulate a request for a decision.”

Takeaway: A concrete setting helps the model reason as it would in a real situation.

5. Action verbs change the dynamics

Principle: The chosen verb determines the type of reasoning expected.

Observed effects

• Explain: Cause-and-effect reasoning.

• Show: Reasoning through examples and illustrations.

• Reveal: Reasoning through investigation or discovery.

Examples

“Explain through three main causes.”
“Show two concrete examples and one counterexample.”
“Reveal the hidden element that changes the conclusion.”

Takeaway: The verb guides the structure of the text and the nature of the reasoning.

6. Silence as a tool

Principle: Leaving some things unspecified gives the model room for interpretation and creativity.

Observed effects

• The model fills in gaps with contextual patterns that are often relevant.

• Too many constraints inhibit thinking and lead to formulaic responses.

Examples

“Give three options, including one bolder option, without justifying them.”
“Write a rough draft without headings, containing only the key ideas.”

Takeaway: Leaving space creates room for creativity.

7. Emotions modulate the energy of the content

Principle: A prompt's emotional tone influences the pace and depth of the response.

Observed effects

- Anger: Direct, sharp, critical writing.
- Sadness: Slow pace, serious tone and introspection.
- Joy: Fast pace, fluid and enthusiastic style.
- Admiration: Expansive, figurative language.

Examples

"Tell me directly what is wrong with this strategy."
"Discuss the systemic risks seriously, in three points."

Takeaway: Emotion structures reasoning by modulating energy and focus.

8. Paradox encourages nuance

Principle: Introducing a contradiction into the prompt pushes the model to produce a nuanced response.

Observed effects

• The model seeks a balance between two logical poles.

• It produces dialectical reasoning rather than simple commentary.

Examples

“Explain why AI is both a threat and a promise.”
“Show how progress can be both a risk and an opportunity.”

Takeaway: Paradox encourages nuance and deeper reflection.

Current limitations of LLMs: two major dimensions

1. Hallucinations

- Generating inaccurate or fabricated content, sometimes with great confidence.
- Difficulty distinguishing truth from plausibility, especially outside the training domain.
- Consequences: Misinformation risks, reduced reliability and the need for human verification.

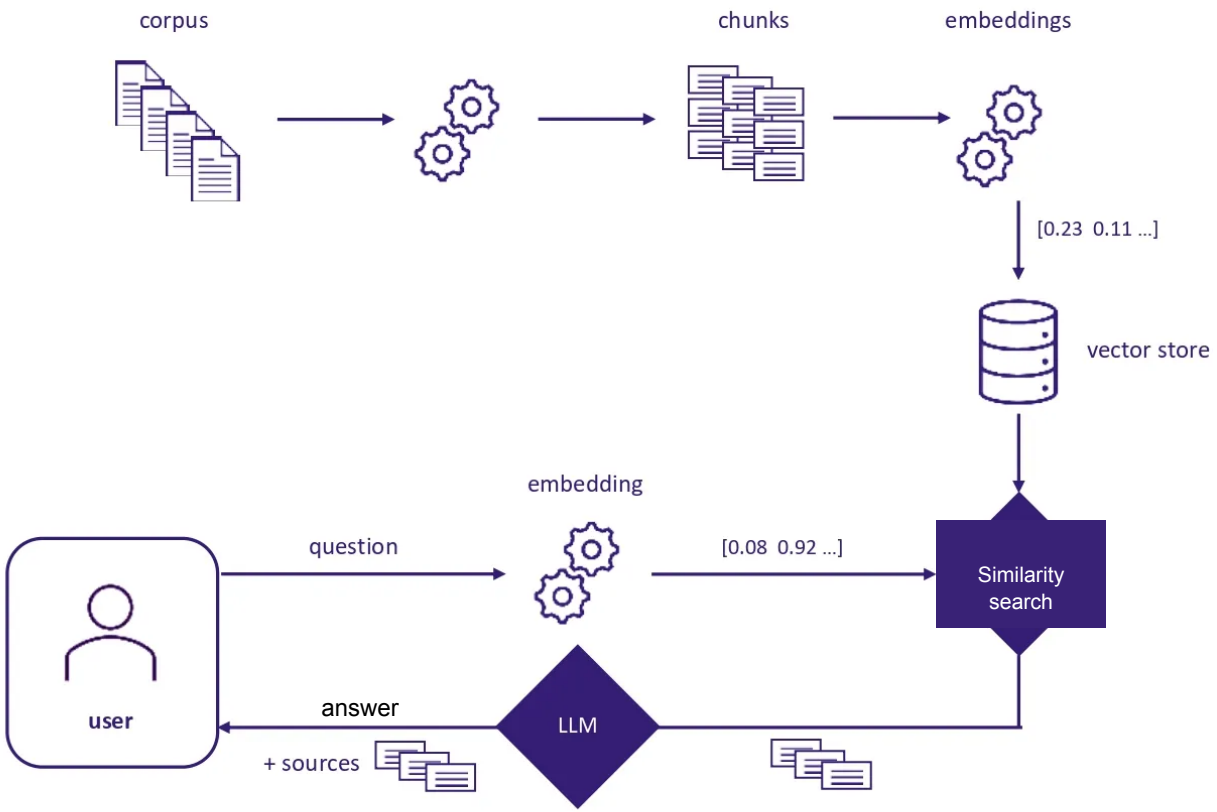
2. Non-determinism

- The same prompt can produce different responses.
- Results are sensitive to temperature, context and small changes in wording.
- A challenge for reproducibility and quality control in LLM-based systems.

Retrieval Augmented Generation (RAG)

RAG principles

Basic RAG



Chunking strategies

Definition: Chunking divides a document into pieces (chunks) to optimize information retrieval and generation in a RAG system.

Main strategies:

- **Fixed-size chunking:** Splits text into fixed-length segments (e.g. 512 tokens). Simple, but may fragment context.
- **Overlapping chunks:** Overlap between segments preserves context and avoids losing critical information.
- **Semantic chunking:** Segmentation based on text structure (paragraphs, headings, sections) or semantic analysis.
- **Recursive chunking:** Progressive splitting according to the granularity of relevant information.

Example tools: LangChain, NLTK, spaCy, tiktoken.

Benefits: Better retrieval, less noise and more efficient use of generative models.

Vector databases

Definition: Store and index numerical representations (embeddings) for efficient semantic search.

Principle: Each document is transformed into a dense vector in a high-dimensional space. Search relies on similarity (cosine, ANN, etc.) rather than keywords.







Advanced RAG approaches

- **Reranking:** Improve the selection of retrieved documents using a more powerful model (e.g. cross-encoder, Cohere Rerank, ColBERT).
- **Retriever optimization:** Improve document indexing and scoring using data representative of the application domain.
- **Graph RAG:** Use a knowledge graph to structure relationships between documents and improve information retrieval.
- **Agent RAG:** Integrate conversational agents able to perform actions (multiple searches, answer verification) before generating a final response.

Reranking in RAG

Definition: Reranking reorders retrieved documents to improve relevance before passing them to the generative model.

Main methods:

- **BM25 + reranker:** Lexical search followed by a neural model that refines the document ranking.
- **Cross-encoder:** A BERT-like model evaluates each document using both the query and the content.
- **Dense reranking:** Adjusts document scores using embeddings learned from domain-specific data.

Example tools: Cohere Rerank, ColBERT, MonoT5, Rank-BERT.

Benefits: More precise results, less noise and better generative-model performance.

Graph RAG

Definition: Graph RAG uses a graph structure to organize and retrieve information in a more contextual, structured way.

Key principles:

- **Relationships between documents:** Nodes represent documents, concepts or entities; edges define their connections.
- **Connectivity-based search:** Retrieval explores relationships between nodes to find the most relevant answers.
- **Context augmentation:** The model can use semantic paths through the graph to enrich the prompt with more coherent data.

Example tools: Neo4j, NetworkX, Weaviate.

Benefits: Better knowledge organization, more accurate retrieval and fewer contextual errors.

Agent RAG

Definition: Agent RAG integrates an autonomous agent that orchestrates document retrieval and use to improve answer generation.

Key principles:

- **Adaptive multiple searches:** The agent runs several queries based on context and dynamically adjusts the retriever.
- **Verification and correction:** The agent can analyze the generated answer, detect potential errors and reformulate the query if necessary.
- **Specific actions:** The agent can interact with databases and external APIs, or execute code to enrich the final answer.

Example tools: LangChain Agents, CrewAI, etc.

Benefits: Greater autonomy, more dynamic retrieval and fewer generation errors.

LLM-based agents

Introduction to agents

Definition: LLM-based agents are systems that can take actions autonomously using the tools available to them.

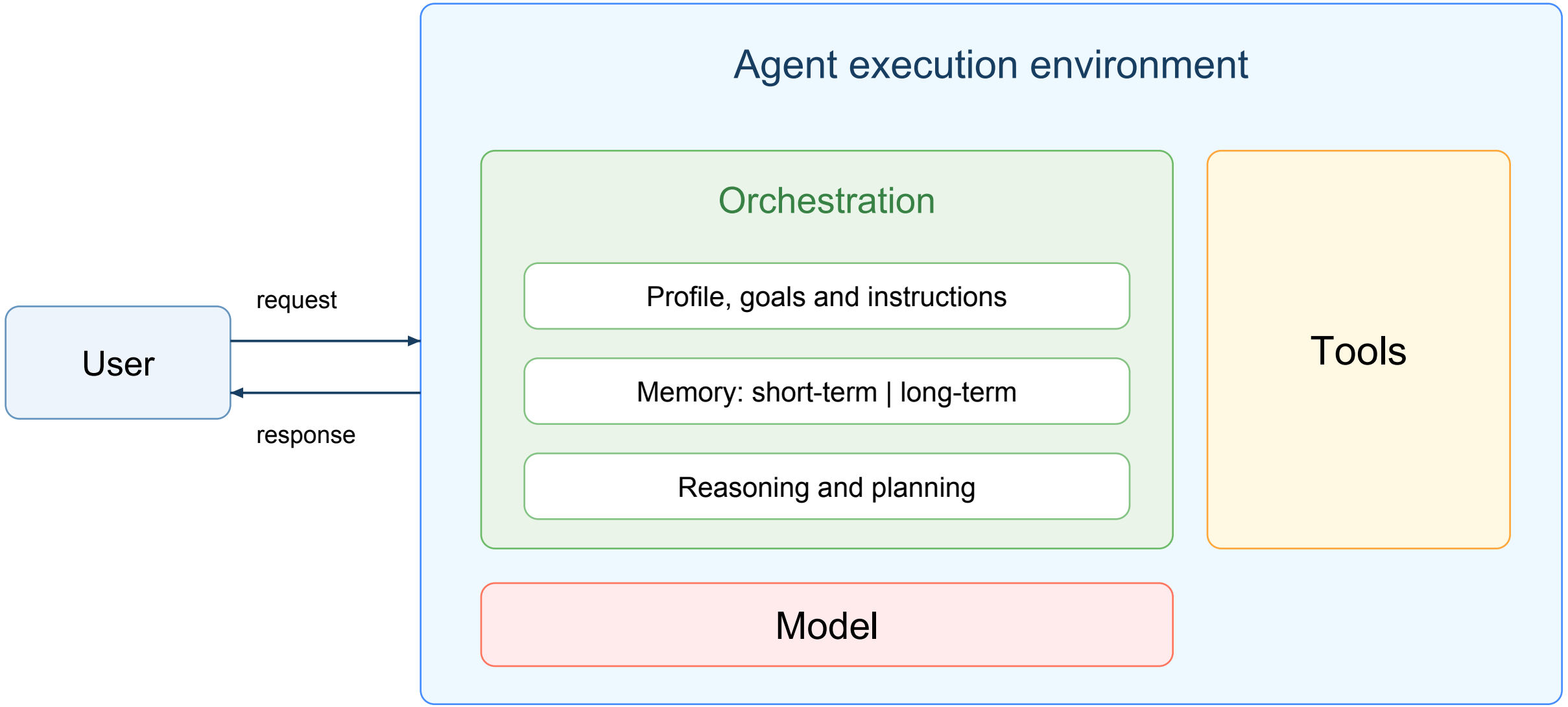
Why agents?

- Language models alone are limited by their training data.
- Agents extend these capabilities using external tools.

Presentation objectives:

- Explain the role of agents in AI.
- Present their architecture and components.
- Explore use cases and modern orchestration techniques.

How an agent works



Chain-of-Thought (CoT)

What is CoT?

A reasoning technique that allows language models to break a complex task into intermediate steps.

How it works:

- The model generates a sequence of explicit reasoning steps before providing an answer.
- It can improve the accuracy and transparency of an agent's decisions.

Applications:

- Solving complex mathematical problems.
- Text comprehension involving logical inference.
- More accurate and consistent responses in AI dialogues.

ReAct: Reasoning + Acting

Definition: ReAct combines reasoning and action to improve decision-making by language models.

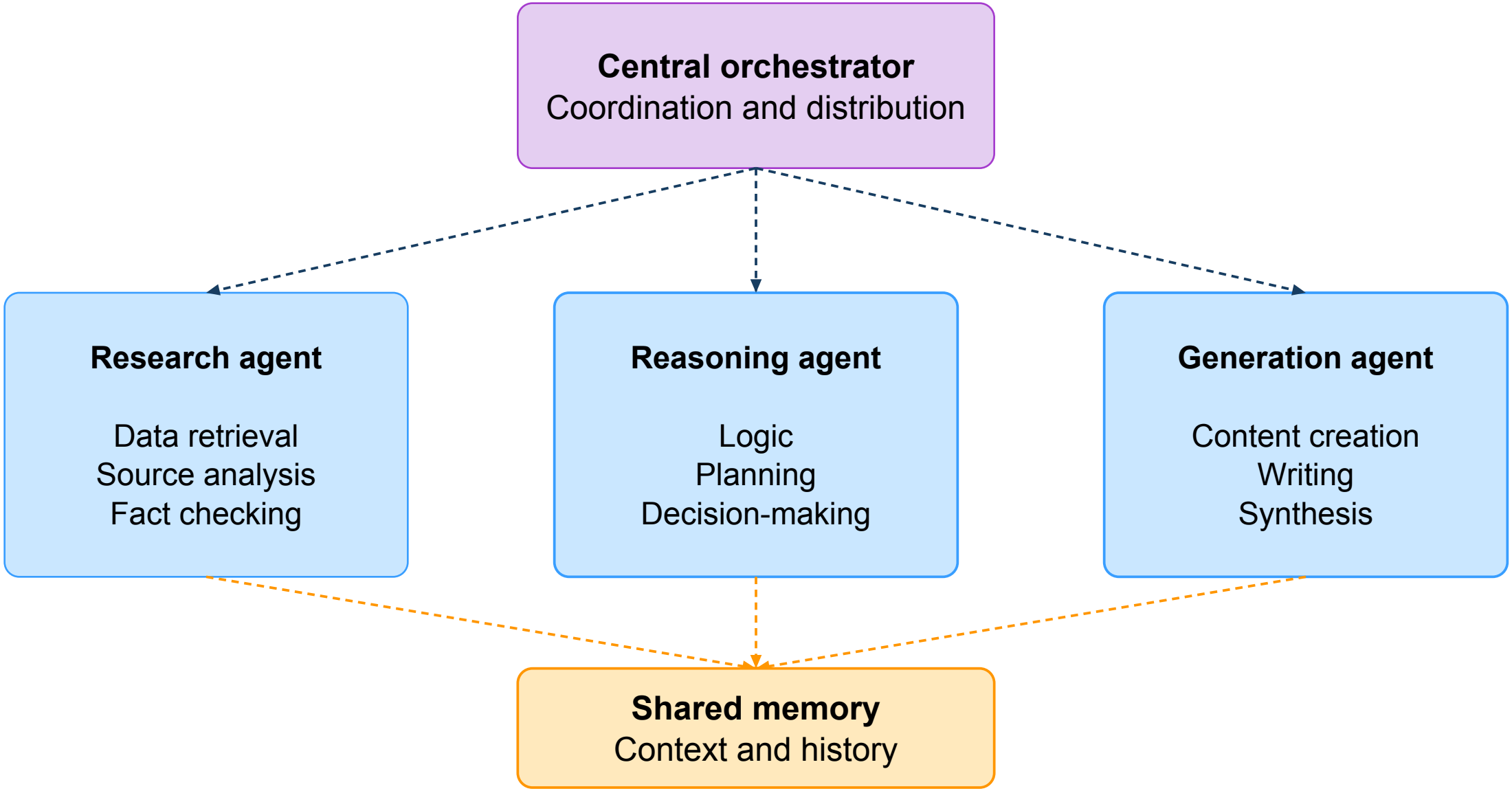
Principle:

- The model observes user input and generates an explicit thought (reasoning).
- It selects an action based on that thought and executes it (acting).
- The action returns an observation, which provides context for the next iteration.
- This reasoning-and-action cycle continues until a satisfactory answer is reached.

Benefits:

- Fewer hallucinations through real-time information.
- Greater interpretability through reasoning traces.
- Better adaptation to dynamic environments.

LLM multi-agent systems



Model Context Protocol (MCP)

Model Context Protocol (MCP) is a client/server protocol that formalizes an LLM's access to external capabilities.

Roles

- **LLM:** Reasoning and action selection.
- **MCP client:** Orchestration, validation and policies.
- **MCP server:** Exposing and executing capabilities.

Fundamental principle:
The LLM only chooses within an explicitly declared action space.

MCP does not change an LLM's ability to reason; it changes the **structure** of what it is allowed to do.

Formal model: a constrained action space

Let the set of actions available to the LLM be:

$A = \{a_1, ..., a_n\}$

Each action a_i (tool) is defined by a contract:

$a_i = (name_i, \Sigma_i^{in}, \Sigma_i^{out}, policy_i)$

- $\boldsymbol{\Sigma^{in}}$**:** Input schema (types, constraints, required fields).
- $\boldsymbol{\Sigma^{out}}$**:** Output schema (result, errors).
- **policy:** Permissions, scopes and budgets.

The LLM must produce a valid pair (a_i, x), where $x \in \Sigma_i^{in}$. Any output outside the contract cannot be executed.

MCP building blocks: Resources, Tools, Prompts

An MCP server exposes three distinct categories:

Resources

• Addressable context (URI).

• Controlled reading: Files, web pages, tables and logs.

Tools

• Executable functions.

• Strictly typed interfaces (JSON Schema).

• Real effects: I/O, APIs and automation.

Prompts

• Parameterized reasoning templates.

• Standardized behaviour across agents.

MCP exposes contracts rather than implementations.

MCP execution cycle: reasoning → action

Minimal sequence:

1. **Discovery**
list_tools, list_resources, list_prompts

2. **Selection**
The LLM chooses a tool and supplies arguments that conform to the schema.

3. **Validation**
The client validates types, constraints and policies.

4. **Execution**
The server executes the action and returns a structured result.

System robustness relies on valid schemas, not on interpreting text.

What MCP actually provides

What it provides

- An explicit action space.
- Separation of decision-making and execution.
- Governance by design (permissions, budgets).
- Auditability through tool calls.

What it does not provide

- More intelligence.
- More autonomy.
- Execution inside the model.

MCP does not make LLMs more powerful; it makes agentic systems easier to control.

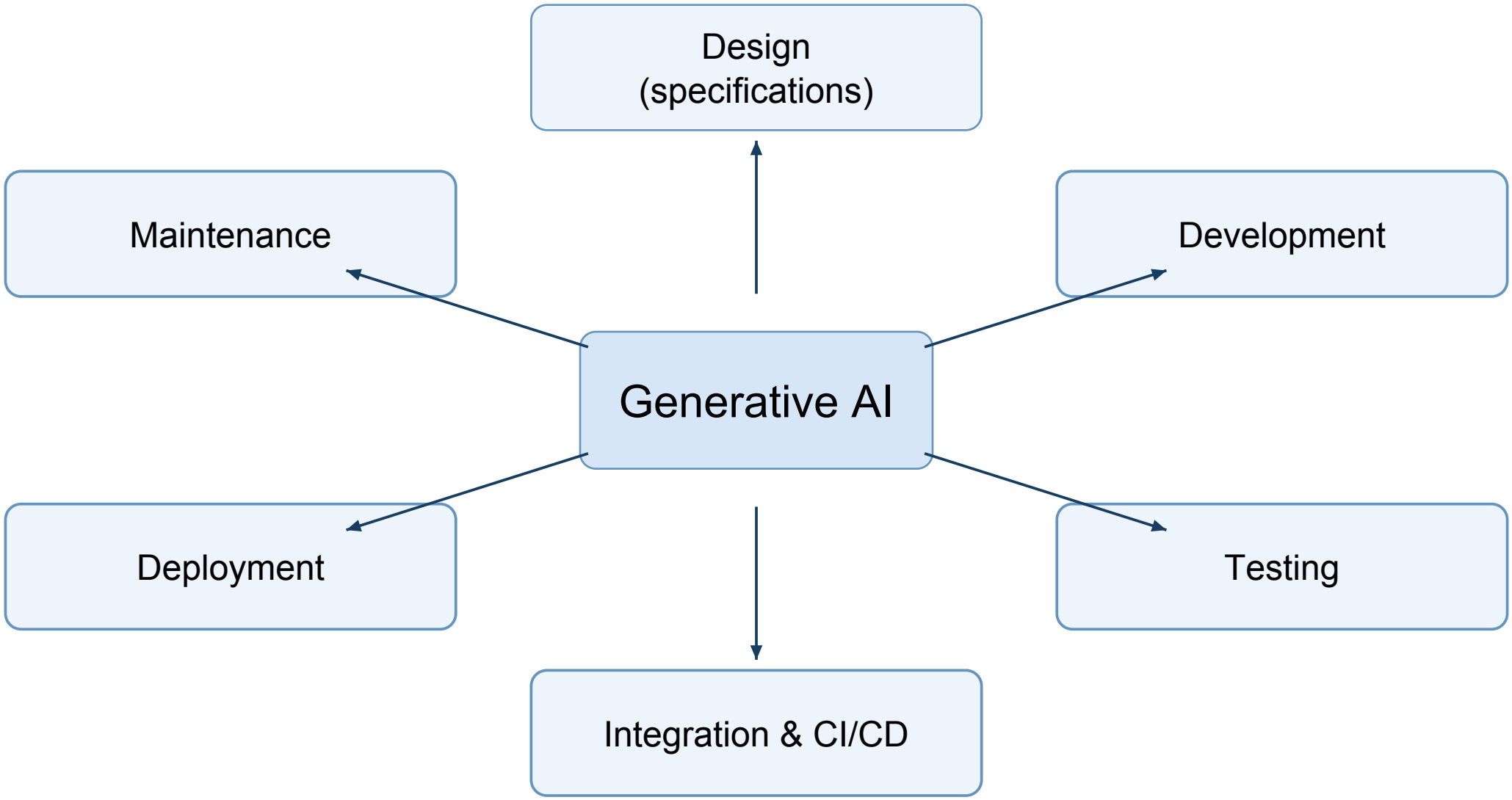
Generative AI in the development lifecycle

Generative AI in the software lifecycle

Generative AI goes beyond code completion: it can support teams at every stage of the software lifecycle.

- **Design:** Clarifying requirements, specifications and architecture sketches.
- **Development:** Code generation and refactoring; help with design patterns.
- **Testing:** Generating unit and functional tests; identifying edge cases.
- **Integration & CI/CD:** Automated PR reviews, changelogs and documentation.
- **Deployment:** Release notes and infrastructure-as-code assistance.
- **Maintenance:** Ticket classification, suggested fixes and documentation derived from code.

Generative AI in the software lifecycle



Design and specifications

Objective: Reduce ambiguity and accelerate the transition from business requirements to a technical solution.

- Turn a user story into API endpoints (routes, verbs and JSON schemas).
- Generate an initial architecture sketch (services, flows and dependencies).
- Produce a glossary or testable acceptance criteria.

Example prompt: “From this user story, propose REST endpoints, error codes and a JSON schema for POST /orders.”

Development

Objective: Accelerate implementation while improving readability and maintainability.

- Generate functions from natural-language descriptions.
- Refactor: Extract helper functions, rename variables and simplify structures.
- Suggest design patterns suited to the context.

Example: Before: A monolithic 80-line function. After: Separate validate(), transform() and persist() functions, with docstrings and typed exceptions.

Testing

Objective: Improve coverage and detect edge cases during development.

- Generate unit tests directly from code.
- Suggest extreme cases: Boundary values, network errors and unexpected formats.
- Create functional scenarios derived from user stories.

Example prompt: “Here is the function price(order). Generate 8 pytest unit tests covering discounts, VAT, unknown currency, negative quantity, thresholds and null values.”

Integration and CI/CD

Objective: Add analysis and generation capabilities to continuous integration pipelines.

- Automated pull request reviews: Detect security flaws, duplication and inconsistent style.
- Automatically generate additional tests to validate commits.
- Produce documentation and changelogs from commit messages.

Example: A GitHub Actions workflow sends the diff to an LLM, which comments on the PR with suggestions.

Deployment

Objective: Automate and simplify production deployment tasks.

- Help write infrastructure-as-code scripts (Terraform, Ansible).
- Automatically generate readable release notes from commits.
- Support DevOps teams in documenting and communicating changes.

Example: From technical commits, generative AI writes a clear changelog for end users.

Maintenance and support

Objective: Resolve problems faster and reduce technical debt.

- Automatically classify incident tickets by priority and category.
- Propose initial fixes for certain bugs.
- Generate documentation from existing code or logs.

Example: An error log is submitted to generative AI, which proposes a diagnosis and a possible fix.

AGENTS.md and SKILL.md files

AGENTS.md

Project-specific instructions: conventions, test commands and contribution rules.

project/AGENTS.md

```
# Validation

- Run tests: pytest

- Add a regression test
for every bug fixed.
```

SKILL.md

A specialized procedure loaded for the task. It may include scripts and references.

code-review/SKILL.md

```
---
name: code-review
description: Review the code.
---

Examine the diff and the tests.
```

The agent applies the instructions and performs actions using its tools.

Four skill repositories to know

Ready-to-use procedures, chosen according to the task.

Addy Osmani
addyosmani/agent-skills

Scope, develop and verify a project.
Specifications, testing and code review.

Anthropic
anthropics/skills

Create and process documents.
PDF, Word, Excel and PowerPoint.

Vercel Labs
vercel-labs/agent-skills

Improve React / Next.js applications.
Interface performance and quality.

Superpowers
obra/superpowers

Organize a coding agent’s work.
Planning, testing and systematic debugging.

Ethics, security and compliance in generative AI

Why discuss ethics and security?

Generative AI raises more than technical challenges: it creates critical issues for organizations.

- **Security:** Risks of secret leakage and prompt-based attacks.
- **Privacy:** GDPR compliance and protection of sensitive data.
- **Intellectual property:** Ownership of training data and AI-generated content.
- **Ethics:** Bias, fairness and responsible use.

Key message: Adopting generative AI requires appropriate governance.

Generative AI security

- **Data leakage:** Sending confidential documents, proprietary code or credentials to an external service; disclosing information to an unauthorized user.
- **Prompt injection:** Malicious instructions in a query, document or web page attempt to redirect the system’s behaviour.
- **Dangerous outputs:** Generating vulnerable code or risky commands that are then used without sufficient verification.
- **Uncontrolled actions:** A system connected to tools may modify, delete or transmit data beyond what was intended.

Protective measures: Limit access and permissions, protect secrets, verify results before execution, require approval for sensitive actions, and provide logging and a stop mechanism.

Privacy issues

Central issue: Generative AI often processes sensitive or personal data.

- **GDPR:** Right to erasure, consent and data minimization.
- **Data location:** EU vs non-EU data centres.

Solutions:

- Anonymize data before sending it.
- Host open-source models internally.
- Use an enterprise proxy.

Intellectual property and generative AI

- **Training data:** Using protected works raises questions about respect for rights, transparency about sources and compensation for creators.
- **Protection of generated content:** In France and the European Union, copyright protection depends on original human creative choices. Using AI therefore does not automatically confer copyright on the output.
- **Risk of reproduction:** Generated text, images or code may reproduce protected elements. AI generation does not guarantee that content can be freely reused.
- **Licences and contracts:** Distinguish between model terms of use, rights to supplied content and conditions for exploiting outputs.

Good practices: Check applicable rights and licences, retain sources and human contributions, and review content before publication or delivery.

Bias and generative AI

- **Reproducing stereotypes:** Models may reproduce or amplify social, cultural or gender prejudices present in their training data.
- **Discriminatory treatment:** Comparable situations may receive different responses based on personal characteristics or indirect cues, such as a first name, an address or a way of speaking.
- **Unequal performance:** Some languages, populations or situations are less well represented in the data. Good average performance may conceal more frequent errors for certain users.
- **Alignment and explicit injection of values:** Alignment and system instructions steer responses according to norms and values chosen by designers or deploying organizations. These choices are not necessarily shared by all users.

Responsibility and generative AI

- **Diluted responsibility:** When an error occurs, each party may shift responsibility to another (model provider, organization or user), leaving no one accountable for the consequences.
- **Overconfidence:** A convincing response may be accepted without verification. Human validation then becomes a formality that lets errors pass.
- **Hard-to-justify decisions:** Missing records of the information used and trade-offs made complicate explaining a decision and identifying the causes of harm.

Good practices: Clarify responsibilities, provide meaningful human oversight, document important decisions, and establish procedures for contesting and correcting outcomes.

Sustainability and generative AI

- **Energy consumption:** Training and using models requires computing resources. Associated emissions depend on usage volume, infrastructure efficiency and the source of electricity.
- **Hardware footprint:** Manufacturing chips, servers and infrastructure requires raw materials. Equipment replacement increases extraction needs and electronic waste.
- **Rebound effect:** More efficient, less expensive models may encourage more use. Savings per request may be offset or exceeded by increased total volume.

Good practices: Prefer lightweight models when suitable, limit input and output tokens to what is needed, avoid redundant calls and reuse relevant results while maintaining the required quality.

Employment, work and skills

- **Job insecurity:** Automating some tasks may reduce staffing needs in certain roles and change the skills sought, with different consequences across occupations and qualification levels.
- **Skills erosion:** Excessive delegation may weaken the ability to understand, produce and verify work. For beginners, it may remove learning opportunities essential to developing expertise.
- **Work intensification:** Time savings may translate into higher targets and a faster pace rather than improved working conditions.
- **Unequal sharing of gains:** Productivity benefits may be concentrated among certain stakeholders, while employees bear the adaptation effort and the risk of replacement.

Sovereignty and generative AI

- **Technological dependence:** Relying on external suppliers can make an organization dependent on their decisions about prices, features, terms of use and service continuity.
- **Vendor lock-in:** Proprietary integrations and specialized investments can make switching providers costly and complex, reducing freedom of choice.
- **Control of strategic assets:** Outsourcing the processing of data, code and business knowledge may reduce organizational control over information assets and increase reliance on external expertise.
- **Geopolitical and industrial dependence:** Access to models, computing infrastructure and components may be affected by political decisions, trade restrictions or international tensions.

Thank you for your attention

redha.moulla@axia-conseil.com